

些尤为知名的端口。

端口	协议	用途
20, 21	FTP	文件传输
22	SSH	远程登录, Telnet的替代品
25	SMTP	电子邮件
80	HTTP	万维网
110	POP-3	访问远程邮件
143	IMAP	访问远程邮件
443	HTTPS	安全的Web (SSL/TLS之上的HTTP)
543	RTSP	媒体播放控制
631	IPP	打印共享

图 6-34 一些分配的端口号

1024~49 151 之间的其他端口可以通过 IANA 注册, 由非特权用户使用, 但是应用程序可以选择自己的端口号。例如, BitTorrent 对等文件共享应用(非正式的)使用了 6881~6887 端口号, 但也可以运行在其他端口号上。

让 FTP 守护进程在系统启动时关联到 21 号端口, 让 SSH 守护进程在系统启动时关联到 22 号端口等类似的做法是完全可能的。然而, 这样做将会使内存散乱在这些守护进程中, 而且大多数时间这些进程都是空闲的。因此, 一般的做法是让一个守护进程同时关联到多个端口上, 然后等待针对这些端口的第一个入境连接。这个特殊的守护进程在 UNIX 中称为 `inetd` (Internet daemon)。当出现一个入境连接请求, `inetd` 就派生出一个新的进程, 在这个进程中调用适当的守护程序, 然后由这个守护程序来处理连接请求。按照这种方式, 除了 `inetd`, 其他所有守护程序都只在确实有工作需要它们做时才被真正激活。`inetd` 通过一个配置文件知道哪个端口应该使用哪个守护程序。因此, 系统管理员可以这样配置系统使得比较忙的端口(比如 80 端口)使用永久守护程序, 而其他端口则让 `inetd` 来处理。

所有的 TCP 连接都是全双工的, 并且是点到点的。所谓全双工, 意味着同时可在两个方向上传输数据; 而点到点则意味着每个连接恰好有两个端点。TCP 不支持组播或者广播传输模式。

一个 TCP 连接就是一个字节流, 而不是消息流。端到端之间不保留消息的边界。例如, 如果发送进程将 4 个 512 字节的数据块写到一个 TCP 流中, 那么这些数据有可能按 4 个 512 字节块、2 个 1024 字节块、1 个 2048 字节块或者其他方式被递交给接收进程(见图 6-35)。接收端不管多么努力尝试, 都无法获知这些数据被写入字节流时的单元大小。

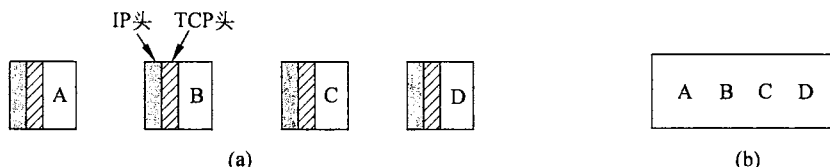


图 6-35

(a) 4 个 512 字节段作为单独的 IP 数据报发送; (b) 在一次 READ 调用中 2048 字节数据被递交给应用程序

UNIX 系统中的文件也具有这样的特性。读文件的程序无法判断该文件是一次写入一块, 还是一次写入一个字节, 或者整个文件被一次性写入的。如同 UNIX 文件一样, TCP 软件不理解字节流的含义, 而且对此也不感兴趣。一个字节就只是一个字节而已。

当一个应用将数据传递给 TCP 时, TCP 可能立即将数据发送出去, 也可能将它缓冲起来 (为了收集更多的数据一次发送出去), 这完全由 TCP 软件自己来决定。然而, 有时候, 应用程序确实希望数据立即被发送出去。例如, 假设一个交互式游戏的用户希望发送一个更新流, 至关重要的是这个流应该被立即发送出去, 而不是缓存起来直到收集到足够的数据后一起发送。为了强制将数据发送出去, TCP 有个 PUSH 标志位概念, 由数据包携带的 PUSH 标志原意是让应用告诉 TCP 不要延迟传输。然而, 应用程序在发送数据时不能从字面上设置 PUSH 标志。相反, 不同的操作系统已经演化出了不同的方案来加速传输 (例如, 在 Windows 和 Linux 的 TCP_NODELAY)。

有关 Internet 的悠长历史, 我们还要提到有关 TCP 服务的一个有趣特性, 该特性还保留在协议中, 但很少被使用, 那就是**紧急数据 (urgent data)**的处理。当一个具有高优先级数据的应用立即被处理时, 例如, 一个交互式用户键入 Ctrl+ C 来中断一个已经开始运行的远程计算时, 发送端应用程序把一些控制信息放在数据流中, 并且将它连同 URGENT 标志一起交给 TCP。这一事件将导致 TCP 停止积累数据, 将该连接上已有的所有数据立即传输出去。

当接收方接收到紧急数据时, 接收端应用程序被中断 (比如, 按 UNIX 的术语是得到了一个信号), 它停止当前正在做的工作, 并且读入数据流以便找到紧急数据。紧急数据的尾部应该被标记出来, 因而应用程序能够知道紧急数据的结束位置。紧急数据的起始处并没有被标记出来, 如何找出紧急数据取决于具体的应用程序。

这个方案提供了一种略微粗糙的信号机制, 而把一切留给应用程序来处理。然而, 尽管紧急数据有潜在的应用价值, 但在早期没有人发现它有令人信服的应用, 因而遭到淘汰。因为这种信号机制让应用程序自己来处理, 现在它的用途因为各种版本实现上的差异而令人气馁。也许未来的传输协议能提供更好的信号机制。

6.5.3 TCP 协议

在这一节, 我们将概述性地介绍 TCP 协议。在下一节我们将逐个讨论 TCP 协议头的每个字段。

TCP 的一个关键特征, 也是主导整个协议设计的特征是 TCP 连接上的每个字节都有它自己独有的 32 位序号。在 Internet 初期, 路由器之间的线路绝大多数是 56 kbps 的租用线路, 所以, 一台满负荷全速运行的主机差不多一周才能遍历完所有的序号。以现代的网络速度, 序号的消耗速度相当惊人, 后面我们将会看到这一点。数据包携带的 32 位序号可用在一个方向上的滑动窗口以及另一个方向上的确认, 正如下面我们讨论的那样。

发送端和接收端的 TCP 实体以段的形式交换数据。TCP 段 (TCP segment) 由一个固定的 20 字节的头 (加上可选的部分) 以及随后 0 个或者多个数据字节构成。TCP 软件决定了段的大小。它可以将多次写操作中的数据累积起来, 放到一个段中发送, 也可以将一次写操作中的数据分割到多个段中发送。有两个因素限制了段的长度。首先, 包括 TCP 头在内的每个段, 必须适合 IP 的 65 515 个字节有效载荷; 其次, 每个网络都有一个**最大传输单元 (MTU, maximum transfer unit)**。发送端和接收端的每个段必须适合 MTU, 才能以单个不分段的数据包发送和接收。实际上, MTU 通常是 1500 字节 (以太网的有效载荷大小),

因此它规定了段长度的上界。

然而，当 IP 数据包穿过一条网络路径，其上某条链路有更小的 MTU 时，还是有可能要对携带 TCP 段的该 IP 数据包实行分段操作。如果这种情况发生，则会降低性能并引起其他问题（Kent 和 Mogul, 1987）。相反，现代 TCP 使用 RFC 1191 给出的技术实现了“路径 MTU 发现”（path MTU discovery）功能，我们在 5.5.5 节对此有过描述。该技术利用 ICMP 错误消息来发现某条路径上任意一条链路的最小 MTU。

TCP 实体使用的基本协议是具有动态窗口大小的滑动窗口协议。当发送端传送一段时，它启动一个计时器。当该段到达接收方时，接收端的 TCP 实体返回一个携带了确认号和剩余窗口大小的段（如果有数据要发送的话，则包含数据，否则就不包含数据），并且确认号的值等于接收端期望接收的下一个序号。如果发送端的计时器在确认段到达之前超时，则发送端再次发送原来的段。

尽管这个协议听起来简单，但是有时候涉及许多非常微妙的细节，后面我们将会介绍。段到达的顺序可能是错误的，所以可能 3072~4095 字节的数据已经到达了，但是它不能被确认，因为 2048~3071 字节还没有到达。段在传输过程中可能会被延迟很长时间，因而发送端超时并重传段。重传的段包含的字节范围有可能与原来传输的段的字节范围不同，所以，这就要求仔细管理，以便跟踪哪些字节已经被正确地接收到了。然而，由于数据流中的每个字节都有它自己唯一的偏移值，所以这项管理工作是可以完成的。

TCP 必须做好处理这些问题的准备，并且以一种有效的方法来解决这些问题。尽管面临各种各样的网络问题，研究人员还是做了相当多的努力来优化 TCP 流的性能。下面将要讨论一些被许多 TCP 实现采用的算法。

6.5.4 TCP 段的头

图 6-36 显示了 TCP 段的结构。每个段的起始部分是一个固定格式的 20 字节头。固定的头部之后可能有头的选项。如果该数据段有数据部分的话，那么在选项之后是最多可达 $65\,535 - 20 - 20 = 65\,495$ 个字节的数据，这里的第一个 20 是指 IP 头，第二个 20 指 TCP 头。没有任何数据的 TCP 段也是合法的，通常被用作确认和控制消息。



图 6-36 TCP 头格式.

现在让我们逐个字段地剖析 TCP 头。源端口(Source port)和目标端口(Destination port) 字段标识了连接的本地端点。TCP 端口加上所在主机的 IP 地址组成了 48 位的唯一端点。源端点和目标端点一起标识了一条连接。这个连接标识符就称为 5 元组(5 tuple)，因为它由 5 个信息组成：协议(TCP)、源 IP 地址和源端口号、目标 IP 地址和目标端口号。

序号(Sequence number)和确认号(Acknowledgement number) 字段执行它们的常规功能。请注意，后者指定的是下一个期待的字节，而不是已经正确接收到的最后一个字节。它是累计确认(cumulative acknowledgement)，因为它用一个数字概括了接收到的所有数据，它不会超过丢失的数据。这两个字段都是 32 位长，因为一个 TCP 流中的每一个数据字节都已经被编号了。

TCP 头长度(TCP header length) 字段指明了 TCP 头包含多少个 32 位的字。这个信息是必需的，因为选项(Options) 字段是可变长的，因而整个头也是变长的。从技术上讲，这个字段实际上指明了数据部分在段内的起始位置(以 32 位字作为单位进行计量)，但因为这个数值正好是按字为单位计量的头长度，所以，两者的效果是等同的。

接下来是长度为 4 位没有被使用的字段。这些位(原先的 6 位中只有 2 位被重新声明使用了)30 年没被使用的事实恰好证明了 TCP 自始至终运行良好。差一点的协议需要这些位来修正原先设计中的错误。

接着是 8 个 1 比特的标志位。当采用 RFC 3168 说明的显式拥塞通知(ECN, Explicit Congestion Notification) 时，CWR 和 ECE 就用作拥塞控制的信号。当 TCP 接收端收到了来自网络的拥塞指示后，就设置 ECE 以便给 TCP 发送端发 ECN-Echo 信号，告诉发送端放慢发送速率。TCP 发送端设置 CWR，给 TCP 接收端发 CWR 信号，这样接收端就知道发送端已经放慢速率，不必再给发送端发 ECN-Echo 信号。我们将在 6.5.10 节中讨论 TCP 拥塞控制中的 ECN 规则。

如果使用了紧急指针(Urgent pointer)，则将 URG 设置为 1。紧急指针指向从当前序号开始找到紧急数据的字节偏移量。这个设施是中断消息的另一种途径。正如我们上面所述，该设施允许发送端以少得不能再少的方式给接收端发信号，无须 TCP 本身卷入中断的事由，但是这种设施很少被用到。

ACK 被设置为 1 表示确认号字段是有效的。几乎所有的数据包都会用到这个标志位。如果 ACK 为 0，则该段不包含确认信息，所以，确认号字段可以被忽略。

PSH 指出这是被推送(PUSH)的数据。特此请求接收端一旦收到数据后立即将数据递交给应用程序，而不是将它缓冲起来直到整个缓冲区满为止(这样做的目的可能是为了提高效率的原因)。

RST 被用于突然重置一个已经变得混乱的连接，混乱有可能是由于主机崩溃，或者其他什么原因造成的。该标志位也可以被用来拒收一个无效的段，或者拒绝一个连接请求。一般而言，如果你得到的段被设置了 RST 位，那说明你遇到问题了。

SYN 被用于建立连接过程。在连接请求中，SYN=1 和 ACK=0 表示该段没有使用捎带确认字段。但是，连接应答捎带了一个确认，因此 SYN=1 和 ACK=1。本质上，SYN 位被用来同时表示 CONNECTION REQUEST 和 CONNECTION ACCEPTED，然后进一步用 ACK 位来区分这两种可能情况。

FIN 被用来释放一个连接。它表示发送端已经没有数据需要传输了。然而, 在关闭一个连接之后, 关闭进程可能会在一段不确定的时间内继续接收数据。SYN 和 FIN 段都有序号, 从而保证了这两种段以正确的顺序被处理。

TCP 中的流量控制是通过一个可变大小的滑动窗口来处理的。窗口大小 (Window size) 字段指定了从被确认的字节算起可以发送多少个字节。窗口大小字段为 0 是合法的, 说明到现在为止已经接收到了多达确认号-1 个字节, 但是接收端没有更多的机会来消耗数据, 希望别再发数据。以后, 接收端可以通过发送一个具有同样确认号但是非零窗口大小字段的段来通知发送端继续发送段。

在第 3 章的协议中, 确认接收到的帧和允许发送新帧是捆绑在一起。这是每个协议采用固定窗口大小的必然结果。在 TCP 中, 确认和允许发送额外数据是完全分离的两种机制。实际上, 接收端可以这样说: “我已经接收到了序号 k 之前的字节, 但是现在我不想要任何更多的数据。” 这种分离 (事实上, 是可变大小的窗口) 带来了格外的灵活性。后面我们将详细地学习这种机制。

校验和 (Checksum) 提供了额外的可靠性。它校验的范围包括头、数据, 以及与 UDP 一样的概念性伪头。除了伪头的协议号为 TCP (6), 并且校验和必需强制执行。有关校验和的详细描述见 6.4.1。

选项 (Options) 字段提供了一种添加额外设施的途径, 主要针对常规头覆盖不到的方面。协议定义了许多选项, 有几个已经被广泛使用。选项的长度可变量, 但必须是 32 位的倍数, 不足部分用零填充。选项可以扩展到 40 个字节, 这是说明的最长 TCP 头。某些选项用在连接建立期间, 主要协商或者通知另一端的能力; 其他选项用在连接生存期间, 通过数据包携带。每个选项具有类型-长度-值 (Type-Length-Value) 编码。

用途最广的选项允许每台主机指定它愿意接受的最大段长 (MSS, Maximum Segment Size)。采用大的段通常比小的段更有效率, 因为这 20 字节头的开销可以分摊到更多的数据上, 但是小型主机可能处理不了大的段。在连接建立过程中, 每一端可以宣布它的最大段长, 并且查看对方给出的最大值。如果一台主机没有使用这个选项, 那么它默认可以接受 536 字节的有效载荷。所有 Internet 主机都要求能够接受 $536+20=556$ 个字节的 TCP 段。两个方向上的最大段长可以不同。

对于具有高带宽、高延迟, 或者两者兼备的线路, 64 KB 的窗口对应于 16 位的字段是个问题。在一条 OC-12 的线路上 (大约 600 Mbps), 只需 1 毫秒就可输出一个完整的 64 KB 窗口。如果往返传播延迟是 50 毫秒 (越洋光缆的典型值), 则发送端将有 98% 空闲, 等待确认的到来。大的窗口允许发送端不停地送出数据。窗口尺度 (Window scale) 选项允许发送端和接收端在连接建立阶段协商窗口尺度因子。双方使用尺度因子将窗口大小字段向左移动至多 14 位, 因此允许窗口最大可达 2^{30} 个字节。大多数的 TCP 实现都支持这个选项。

时间戳 (Timestamp) 选项携带由发送端发出的时间戳, 并被接收端回应。一旦在连接建立阶段启用了它, 那么每个数据包都要包含这个选项, 主要用来计算来回时间样值, 该样值被用在估算多久之后数据包可以被认为丢失。它还可以被用作 32 位序号的逻辑扩展。在一条快速连接上, 序号空间很快回绕到零, 这样将导致分不清新数据包和老数据包。防

止序号回绕（PAWS, Protection Against Wrapped Sequence numbers）方案根据时间戳丢弃入境段，从而解决这个序号回绕问题。

最后，选择确认（SACK, Selective ACKnowledgement）选项使得接收端可以告诉发送端已经接收到段的序号范围。这是对确认号的补充，可用在一个数据包已丢失但后续（或者重复）数据到达的特定情况下。新到达的数据无法反映出头的确认号字段，因为该字段给出的仅仅是下一个期待的有序字节。有了 SACK，发送端可以明显地感知到接收端已经接收了什么数据，并据此确定应该重传什么数据。SACK 由 RFC 2108 和 RFC 2883 定义，并逐渐为人们所应用。我们将在 6.5.10 节的拥塞控制中描述 SACK 的使用。

6.5.5 TCP 连接建立

TCP 使用了三次握手法来建立连接，这个方法我们在 6.2.2 节中讨论过。为了建立一个连接，某一端，比如说服务器，必须先依次执行 LISTEN 和 ACCEPT 原语，然后被动地等待入境连接请求；并且可以指定只接受一个特定的请求源，也可以不指定。

另一端，比如说客户，执行 CONNECT 原语，同时说明它希望连接的 IP 地址和端口、它愿意接受的最大 TCP 段长，以及一些可选的用户数据（比如口令）等参数。CONNECT 原语发送一个 SYN 标志位置为 on 和 ACK 标志位置为 off 的 TCP 段，然后等待服务器的响应。

当这个段到达接收方时，那里的 TCP 实体检查是否有一个进程已经在目标端口字段指定的端口上执行了 LISTEN。如果没有，则它发送一个设置了 RST 的应答报文，拒绝客户的连接请求。

如果某个进程正在该端口上监听，那么，TCP 实体将入境的 TCP 段交给该进程处理。该进程可以接受或者拒绝这个连接请求。如果它接受，则发送回一个确认段。正常情况下，发送的 TCP 段顺序如图 6-37（a）所示。请注意，SYN 段只消耗了 1 个字节的序号空间，所以它可被毫无异议地确认。

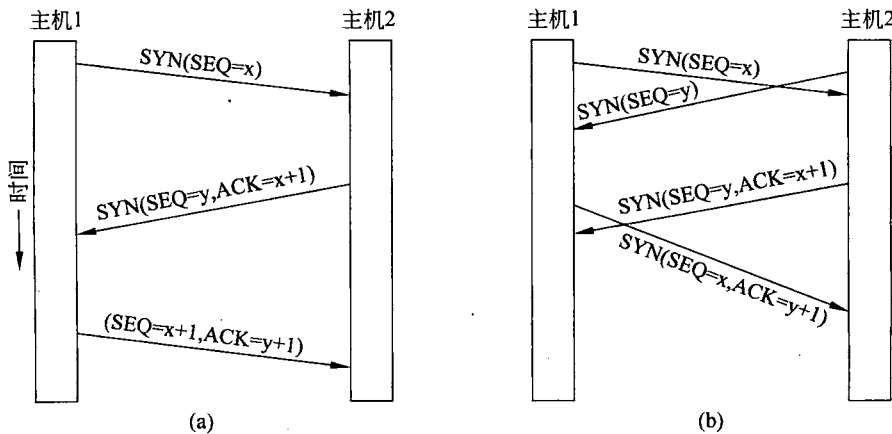


图 6-37

(a) 正常情况下的 TCP 连接建立；(b) 两端同时建立连接的情形

如果两台主机同时企图在一对套接字之间建立连接,则事件序列如图 6-37 (b) 所示。这些事件的结果是恰好只建立了一个连接,而不是两个,因为所有的连接都是由它们的端点来标识的。如果第一个请求产生了一个由(x, y)标识的连接,第二个请求也建立了这样一个连接,那么,实际上只有一个表项,即(x, y)。

回想一下,每个主机选择的初始序号应该循环比较缓慢,而且不能是一个常数,比如 0。这个规则是为了防止被延迟的重复数据包,正如我们在 6.2.2 节所讨论的。最初,采用了一个基于时钟的方案,时钟每 4 微秒滴答一次,由此来解决初始序号问题。

然而,三次握手的实现方式有个漏洞,当监听进程接受一个连接请求,并立即以 SYN 段作为响应时,它必须记住该 SYN 段的序号。这意味着一个恶意的发送端很容易地占据一个主机资源。具体做法是这样的:恶意用户绵绵不绝地发送 SYN 段请求服务器的连接,但又故意不完成连接建立的后续过程,由此可消耗掉一台主机的资源。这种攻击称为 SYN 泛洪 (SYN flood),它在 20 世纪 90 年代致使许多 Web 服务器瘫痪。

抵御这种攻击的一种方法是使用 SYN “小甜饼” (SYN Cookie)。主机不去记忆序号,而是选择一个加密生成的序号,将它放在出境段中,并且忘记它。如果三次握手完成后,该序号 (加 1) 将返回主机。主机运行相同的加密函数,只要该函数的输入是已知的 (例如,其他主机的 IP 地址和端口,和一个本地密钥),它就能重新生成正确的序号。这个过程允许主机检查确认号是否正确,而不必记住单独的序号。这里存在一些注意事项,比如无法处理 TCP 选项,所以只有当主机容易受到 SYN 泛洪时,才可以用 SYN Cookie。然而,这是连接建立过程的一种有趣扭曲。欲了解更多信息,请参阅 RFC 4987 和 (Lemon, 2002)。

6.5.6 TCP 连接释放

虽然 TCP 连接是全双工的,但是,为了理解 TCP 连接是如何释放的,最好将 TCP 连接看成一对单工连接。每个单工连接的释放彼此独立。为了释放一个连接,任何一方都可以发送一个设置了 FIN 标志位的 TCP 段,这表示它已经没有数据要发送了。当 FIN 段被另一方确认后,这个方向上的连接就被关闭,不再发送任何数据。然而,另一个方向上或许还在继续着无限的数据流。当两个方向都关闭后,连接才算被彻底释放。通常情况下,释放一个连接需要 4 个 TCP 段:每个方向上一个 FIN 和一个 ACK。然而,第一个 ACK 和第二个 FIN 有可能被组合在同一个段中,从而将所需段总数降低到 3 个。

正如在电话通话过程中,双方说完再见之后同时挂断电话一样,一个 TCP 连接的两端也可能会同时发送 FIN 段。这两个段按常规的方法被单独确认,然后关闭连接。实际上,两台主机可以先后释放连接,或者同时释放连接,这两者之间并没有本质的区别。

为了避免两军对垒问题 (在 6.2.3 节中讨论过的),需要使用计时器。如果在两倍于最大数据包生存期内,针对 FIN 的响应没有出现,那么 FIN 的发送端直接释放连接。另一方最终会注意到似乎没人在监听连接,因而也会超时。虽然这种方案不是完美无缺,但由于理论上根本不存在完美的解决方案,所以, TCP 也只好这样做了。实际上,这种做法很少产生问题。

6.5.7 TCP 连接管理模型

建立连接和释放连接所需要的步骤可以用一个有限状态机来表示，该状态机的 11 种状态如图 6-38 所列。在每一种状态中，都存在特定的合法事件。当一个合法事件发生时，可能需要采取某个动作。当发生其他事件时，则报告一个错误。

状态	描述
CLOSED	没有活跃的连接或者挂起
LISTEN	服务器等待入境呼叫
SYN RCVD	到达一个连接请求；等待ACK
SYN SENT	应用已经启动了打开一个连接
ESTABLISHED	正常的数据传送状态
FIN WAIT1	应用没有数据要发了
FIN WAIT2	另一端同意释放连接
TIME WAIT	等待所有数据包寿终正寝
CLOSING	两端同时试图关闭连接
CLOSE WAIT	另一端已经发起关闭连接
LAST ACK	等待所有数据包寿终正寝

图 6-38 TCP 连接管理有限状态机使用的状态

每个连接都从 CLOSED 状态开始。当它执行了一个被动打开操作（LISTEN），或者一个主动打开操作（CONNECT）后，它就离开 CLOSED 状态。如果另一端执行了相反的操作，则连接就建立起来，当前状态变成 ESTABLISHED。连接的释放过程可以由任何一方发起。当释放完成时，状态又回到 CLOSED。

有限状态机本身如图 6-39 所示。在图中用粗线表示客户主动连接到一个被动服务器上的一般情形，其中客户部分用的是实线，服务器部分用的是虚线。细线表示了不常用的事件序列。图 6-39 中的每条线都标记成一对“事件/动作”（event/action）。这里的事件既可以是由用户发起的系统调用（CONNECT、LISTEN、SEND 或者 CLOSE），也可以是到达了一个段（SYN、FIN、ACK 或者 RST），或者是发生了两倍于最大数据包生存期的超时事件。动作可以是发送一个控制段（SYN、FIN 或者 RST），或者什么也不做（在图中标记为-）。括号中显示的是说明。

为了更好地理解这个图，你可以首先沿着客户的路径（粗实线），然后沿着服务器的路径（粗虚线）来查看。当客户机器上的一个应用程序发出 CONNECT 请求，本地的 TCP 实体创建一条连接记录，并将它标记为 SYN SENT 状态，然后发送一个 SYN 段。请注意，在一台机器上可能同时有许多个连接处于打开（或者正在被打开）状态，它们可能代表了多个应用程序，所以，状态是针对每个连接的，并且每个连接的状态被记录在相应的连接记录中。当 SYN+ACK 到达的时候，TCP 发出三次握手过程的最后一个 ACK 段，然后切换到 ESTABLISHED 状态。现在可以发送和接收数据了。

当一个应用结束时，它执行 CLOSE 原语，从而使本地的 TCP 实体发送一个 FIN 段，并等待对应的 ACK（虚线框标记了“主动关闭”）。当 ACK 到达时，状态迁移到 FIN WAIT 2，而且连接的一个方向被关闭。当另一方也关闭时，会到达一个 FIN 段，然后它被确认。现在，双方都已经关闭了连接，但是，TCP 要等待一段长度为最大数据包生存期两倍的时

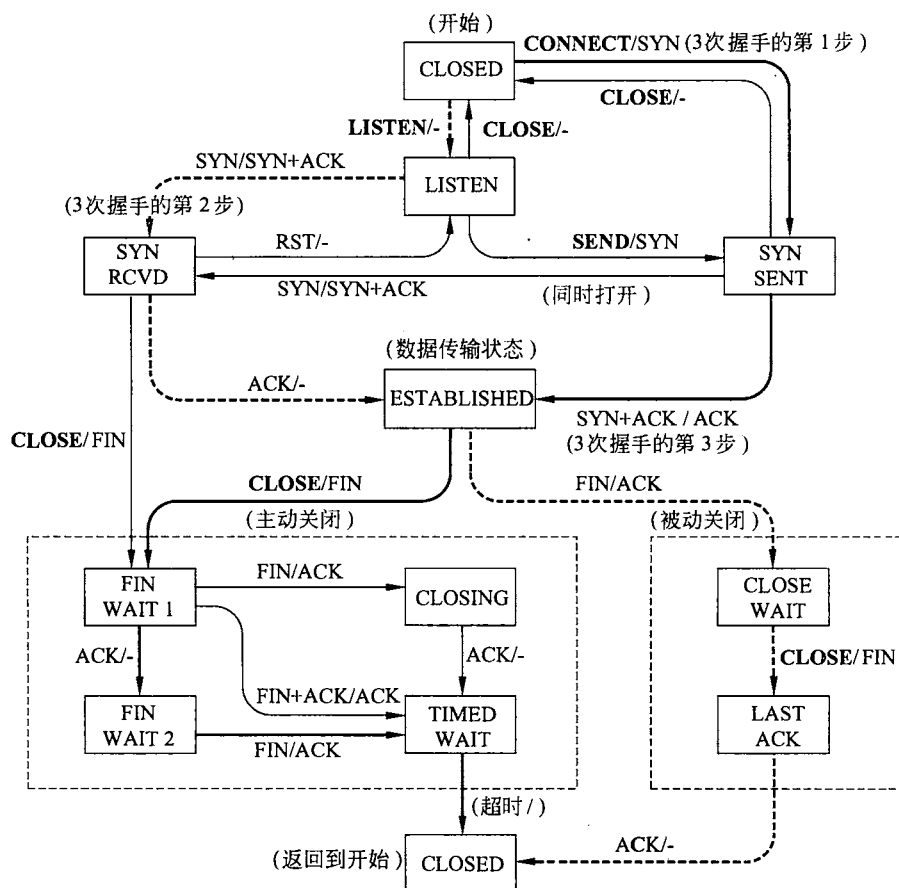


图 6-39 TCP 连接管理有限自动机

粗实线是客户端的正常路径，粗虚线是服务器端的正常路径。细线是不常发生的事件。每次状态迁移由引发的事件以及相应动作标记，事件和动作斜杠分割

间，才能确保该连接上的所有数据包都已寿终正寝，以防万一发生确认被丢失的情形。当计时器超时后，TCP 删除该连接记录。

现在让我们从服务器的角度来看连接管理的情况。服务器执行 LISTEN，并等待入境连接请求。当收到一个 SYN 时，服务器就确认该段并且进入到 SYN RCVD 状态。当服务器本身的 SYN 被确认后，就标志着三次握手过程的结束，服务器进入到 ESTABLISHED 状态。从现在开始双方可以传输数据了。

当客户完成了自己的数据传输，它就执行 CLOSE，从而导致 TCP 实体发送一个 FIN 到服务器（虚线框标记了“被动关闭”）。然后，服务器接到信号；当它也执行了 CLOSE 时，TCP 实体给客户发送一个 FIN 段。当该段的来自客户的确认返回后，服务器释放该连接，并且删除相应的连接记录。

6.5.8 TCP 滑动窗口

正如前面所提到，TCP 的窗口管理将正确接收段的确认和接收端的接收缓冲区分配分离开来。例如，假设接收端有一个 4096 字节的缓冲区，如图 6-40 所示。如果发送端传送

了一个 2048 字节的数据段，并且该数据段已被正确地接收，那么，接收端将确认该数据段。然而，由于接收端现在只剩下 2048 字节的缓冲区空间（在应用程序从缓冲区中取走数据之前），所以它将宣告下一个期望字节开始窗口为 2048。

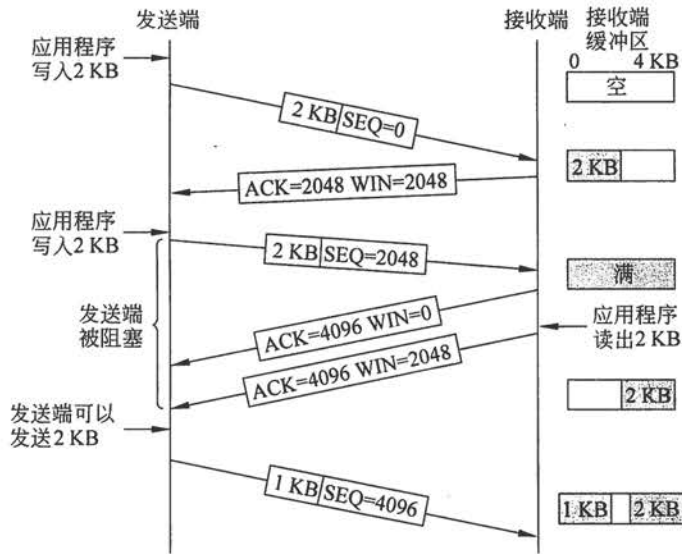


图 6-40 TCP 的窗口管理

现在发送端又传输了另一个 2048 字节长的段，该段被确认了，但是接收端宣告的窗口大小变成了 0。因此，发送端不得不停止，等待接收主机上的应用进程从缓冲区中取走一些数据。到那时候，TCP 实体可以向发送端宣告一个更大的窗口，发送端才能发送数据。

当窗口变为 0 时，发送端不能如通常那样发送段了，但这里有两种意外情形。第一，紧急数据仍可以发送，比如，允许用户杀掉远程机器上运行的某一个进程。第二，发送端可以发送一个 1 字节的段，以便强制接收端重新宣告下一个期望的字节和窗口大小。这种数据包称为窗口探测（window probe）。TCP 标准明确地提供了这个选项，来防止窗口更新数据包丢失后发生死锁。

发送端并不一定一接到应用程序传递来的数据就马上将数据传送出去；同样，接收端也不一定必须尽快地发送确认段。例如，在图 6-40 中，当第一块 2 KB 数据到来时，TCP 知道它有 4 KB 的窗口可以使用，所以它完全可以将这些数据缓冲起来，直到另一个 2 KB 的数据到来后，发送一个包含 4 KB 有效载荷的段。TCP 实体可以充分利用这种自由度来提高传输性能。

考虑一个远程终端连接，比如使用 SSH 或者 Telnet，该连接对用户的每次击键动作都做出响应。在最差的情况下，当一个字符到达发送端的 TCP 实体，TCP 创建一个 21 字节的 TCP 段，并将它交给 IP 组成一个 41 字节的 IP 数据报，然后发送出去；在接收端，TCP 立即发送一个 40 字节的确认（20 字节的 TCP 头加上 20 字节的 IP 头）。以后，当远程终端读取了这个字节之后，TCP 发送一个窗口更新段，它将窗口向前移动 1 个字节。这个数据包也是 40 字节长。最后，当远程终端处理了该字符以后，它发送一个 41 字节的数据包作为该字符的回显。总共累计起来，对于每次敲入的字符，需要使用 162 字节的带宽，并发送 4 个数据段。对于带宽紧缺的场合，这种处理方法显然并不合适。

针对这样的情况,许多 TCP 实现采用了一种称为延迟确认 (delayed acknowledgement) 的优化方法。基本想法是将确认和窗口更新延迟 50 毫秒,希望能够获得一些数据免费搭载过去。假设远程终端在 500 毫秒内回显,则现在远程用户只需要送回一个 41 字节的数据包即可,从而将数据包数和所用带宽减少了一半。

尽管通过延迟确认减少了接收端给予网络的负载,但是发送端发送多个小数据包的工作方式仍然非常低效(比如,41 字节的数据包只包含 1 个字节的数据)。避免这种用法的一种办法是采用 Nagle 算法 (Nagle, 1984)。Nagle 的建议非常简单:当数据每次以很少量方式进入到发送端时,发送端只是发送第一次到达的数据字节,然后将其余后面到达的字节缓冲起来,直到发送出去的那个数据包被确认;然后将所有缓冲的字节放在一个 TCP 段中发送出去,并且继续开始缓冲字节,直到下一个段被确认。这就是说,任何时候只有第一个发送的数据包是小数据包。如果在一个来回时间内应用程序发送了许多数据,那么 Nagle 算法可以将这些数据放置在一个段中发送,由此大大地减少所需的带宽。另外,如果应用传递来的数据足够多,多到可以填满一个最大数据段,则该算法也允许发送一个新的段。

什么时候不用它更好?特别是,在 Internet 上玩互动游戏时,玩家通常需要一个快速的短数据包流。如果把更新收集起来以突发的方式发送将使得游戏的响应不稳定,这显然会引起用户不满。一个更微妙的问题是 Nagle 算法有时可能会延迟确认的相互作用,而造成暂时的死锁:接收端等待(上层)数据到来以便可以捎带确认,而发送端等待确认的到来以便能够发送更多的数据。这种相互作用可能导致延迟网页的下载。由于存在这些问题,可以禁用 Nagle 算法(这就是所谓的 TCP_NODELAY 选项)。(Mogul 和 Minshall, 2001) 讨论了这个问题以及其他的解决方案。

降低 TCP 性能的另一个问题是低能窗口综合症(silly window syndrome)(Clark, 1982)。当数据以大块形式被传递给发送端 TCP 实体,但是接收端的交互式应用每次仅读取一个字节数据的时候,这个问题就会发生。为了看清此问题,请参考图 6-41。初始时,接收端的 TCP 缓冲区为满,发送端知道这一点(即它有一个大小为 0 的窗口)。然后,交互式应用从 TCP 流中读取一个字符,这个动作使得接收端的 TCP 欣喜若狂,它立刻发送一个窗口更新段给发送端,告诉它现在可以发送 1 个字节过来。发送端很感激,立即发送 1 个字节。现在缓冲区又满了,所以,接收端对这 1 字节的数据段进行确认,同时设置窗口大小为 0。这种行为可能会永久地持续下去。

Clark 的解决方案是禁止接收端发送只有 1 个字节的窗口更新段。相反,它强制接收端必须等待一段时间,直到有了一定数量的可用空间之后再通告给对方。特别是,只有当接收端能够处理它在建立连接时宣告的最大数据段,或者它的缓冲区一半为空时(相当于两者之中取较小的值),它才发送窗口更新段。而且,发送端不发送太小的段也会有所帮助。相反,它应该等待一段时间,直到可以发送一个满的段,或者至少包含接收端缓冲区一半大小的段。

Nagle 的算法和 Clark 针对低能窗口综合症的解决方案相互补充。Nagle 试图解决由于发送端应用每次向 TCP 传递一个字节而引起的问题;Clark 则试图解决由于接收端应用每次从 TCP 流中读取一个字节而引起的问题。这两种方案都有效的,而且可以一起工作。发送端的目标是不发送太小的数据段,接收端也不要请求太小的段。

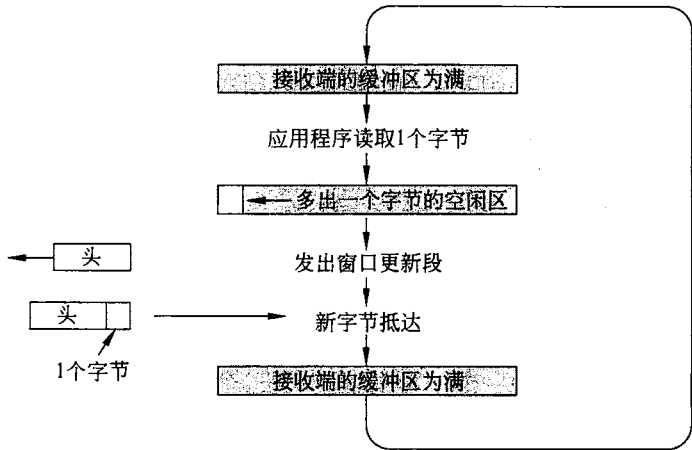


图 6-41 低能窗口综合症

接收端的 TCP 实体除了向发送端宣告较大的窗口以外，还可以进一步提高性能。如同发送端的 TCP 一样，接收端的 TCP 也可以缓冲数据，所以它可以阻塞上层应用的 READ 请求，直至它积累了大块的数据。这样做可以减少调用 TCP 的次数，从而减少额外的开销。当然，这样做也增加了响应的时间，但是，对于像文件传输这样的非交互式应用来说，效率可能比单个请求的响应时间更加重要。

接收端必须处理的另一个问题是乱序到达的数据段。接收端将缓冲这些数据，直至可以按照顺序递交给应用程序为止。实际上，没有什么比丢弃乱序到达的段更糟糕的事情，因为若丢弃这些段，那么它们将被发送端重传，这是一种浪费。

只有当确认字节之前的所有数据都到达之后才能发送确认，这种方式称为累计确认 (cumulative acknowledgement)。如果接收端已经获得段 0、1、2、4、5、6、7，它可以确认直到段 2（包括段 2）之前的数据。当发送端超时，然后重发段 3。因为接收端已经缓冲了段 4~段 7，一旦它收到段 3 就可立即确认直到段 7 的全部字节。

6.5.9 TCP 计时器管理

TCP 使用多个计时器（至少从概念上讲是计时器）来完成它的工作。其中最重要的是重传计时器 (RTO, Retransmission TimeOut)。当 TCP 实体发出一个段时，它同时启动一个重传计时器。如果在该计时器超时前该段被确认，则计时器被停止。另一方面，如果在确认到来之前计时器超时，则段被重传（并且该计时器被重新启动）。于是问题就来了：超时间隔应该设为多长？

这个问题在传输层上比诸如 802.11 那样的数据链路协议要更加困难。在数据链路层，期望的延迟是可以测量的毫秒数，并且是高度可预测的（即方差很小），所以，计时器可以被设置成恰好比期望的确认到达时间稍长即可，如图 6-42 (a) 所示。由于在数据链路层中确认极少被延迟（因为不存在拥塞），所以，如果在预期的时间内确认没有到来，则往往意味着帧或者确认已经被丢失了。

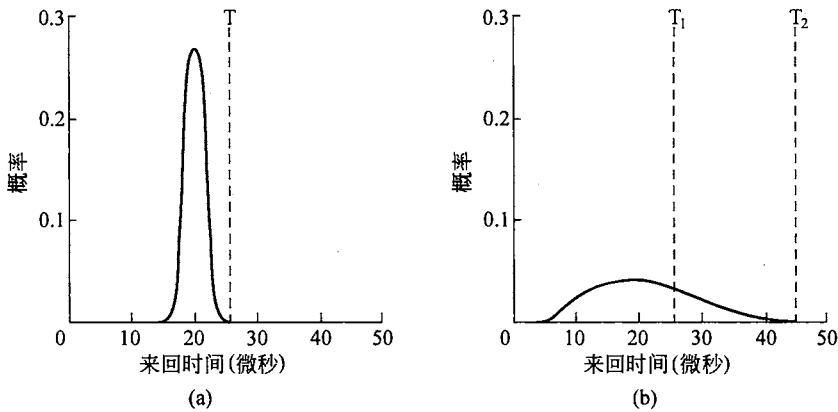


图 6-42

(a) 数据链路层的确认到达时间的概率密度； (b) TCP 确认到达时间的概率密度

TCP 面临着截然不同的环境。TCP 确认回到发送端所需时间的概率密度函数看起来更像 6-42 (b)，而不是图 6-42 (a)，它更大也更可变。要想确定到达接收方的往返时间非常棘手，即使知道了这段时间，要确定超时间隔也非常困难。如果超时间隔设置得太短，比如说图 6-42(b)中的 T_1 ，则会发生大量不必要的重传，这么多无用数据包反而堵塞 Internet；如果超时间隔被设置得太长（比如 T_2 ），则一旦数据包丢失之后，由于太长的重传延迟，所以性能会受到影响。而且，确认到达时间分布的均值和方差也会随着拥塞的发生或者解决而在几秒钟时间内迅速地改变。

解决方案是使用一个动态算法，它根据网络性能的连续测量情况，不断地调整超时间隔。TCP 通常采用的算法是 (Jacobson, 1988) 算法，其工作原理如下所述。对于每一个连接，TCP 维护一个变量 SRTT (Smoothed Round-Trip Time, 平滑的往返时间)，它代表到达接收方往返时间的当前最佳估计值。当一个段被发送出去时，TCP 启动一个计时器，该计时器有两个作用，一是看该段被确认需要多长时间；二是若确认时间太长，则触发重传动作。如果在计时器超时前确认返回，则 TCP 测量这次确认所花的时间，比如说 R。然后它根据下面的公式更新 SRTT：

$$SRTT = \alpha SRTT + (1 - \alpha) R$$

这里 α 是一个平滑因子，它决定了老的 RTT 值所占的权重。典型情况下 $\alpha=7/8$ 。这类公式是指数加权移动平均 (EWMA, Exponentially Weighted Moving Average)，或者丢弃样本中噪声的低通滤波器。

即使有了一个好的 SRTT 值，要选择一个合适的重传超时间隔仍然不是一件容易的事情。在最初的实现中，TCP 使用 $2 \times SRTT$ ，但经验表明常数值太不灵活，当发生变化时它不能够很好地做出反应。尤其是，随机流量的排队模型（即泊松分布）预测的结果是当负载接近容量时，延迟不仅变大，而且变化也大。这可能导致重传计时器超时而重新传输一个数据包的副本，尽管原先的数据包仍然在网络中传输着。这在高负载的情况下也有可能发生，最糟糕的是将额外的数据包发到网络中。

为了解决这个问题，Jacobson 提议让超时值对往返时间的变化以及平滑的往返时间要变得敏感。这种改变要求跟踪另一个平滑变量，RTTVAR (往返时间变化, Round-Trip Time

VARiation), 即更新为下列公式:

$$RTTVAR = \beta RTTVAR + (1 - \beta) |SRTT - R|$$

这就像以前的 EWMA, 典型地, $\beta = 3/4$ 。因此, 重传超时值, RTO 为

$$RTO = SRTT + 4 \times RTTVAR$$

这里选择因子 4 多少有点随意, 但是乘以 4 的操作用一个移位操作就可以实现, 而且, 所有数据包中大于标准方差 4 倍以上的不足总数的 1%。注意 RTTVAR 并不是确切地等于标准方差 (真正的它是平均方差), 但它实际上足够接近。Jacobson 文章充满着一些聪明的技巧, 使用整数加法、减法和移位操作来计算超时值。这种经济运算对于现代主机并不是很需要, 但它已成为文化的一部分, 使得 TCP 可运行在所有种类的设备上, 从超级计算机到微型设备。到目前为止, 还没有人把它搬到 RFID 上, 但这天会来吗? 天知道!

计算超时值的详细过程, 包括变量的初始值设置, 可参考 RFC 2988。重传计时器的最小值为 1 秒, 无论估算值是多少。这是为了防止根据测量获得的欺骗性重传值而选择的保守值。

在采集往返时间的样值 R 过程中有可能引发的一个问题是, 当一个段超时并重新发送以后该怎么办? 确认到达时, 无法判断该确认是针对第一次传输, 还是针对后来的重传。若猜测错误, 则会严重影响重传超时值。Phil Karn 发现这个问题很艰难。Karn 是一名业余无线电爱好者, 他对在一种极不可靠的无线介质上传输 TCP/IP 数据包有浓厚的兴趣。他提出了一个很简单的建议: 不更新任何重传段的估算值。此外, 每次连续重传的超时间隔值加倍, 直到段能一次通过为止。这个修正算法称为 **Karn 算法** (Karn 和 Partridge, 1987)。大多数 TCP 实现都采用了此算法。

重传计时器并不是 TCP 使用的唯一计时器。第二个计时器是**持续计时器** (persistence timer)。它的设计意图是为了避免出现以下所述的死锁情况。接收端发送一个窗口大小为 0 的确认, 让发送端等一等。稍后, 接收端更新了窗口, 但是, 携带更新消息的数据包丢失了。现在, 发送端和接收端都在等待对方的进一步动作。当持续计时器超时后, 发送端给接收端发送一个探测消息。接收端对探测消息的响应是将窗口大小告诉发送端。如果它仍然为 0, 则重置持续计时器, 并开始新一轮循环。如果它非 0, 则现在可以发送数据了。

有些 TCP 实现使用了第三个计时器, 即**保活计时器** (keepalive timer)。当一个连接空闲了较长一段时间以后, 保活计时器可能超时, 从而促使某一端查看另一端是否仍然还在。如果另一端没有响应, 则终止连接。这个特性是有争议的, 因为它增加了额外的开销, 而且有可能由于瞬间的网络分区而终止掉一个本来很正常的连接。

每个 TCP 连接使用的最后一个计时器被应用于连接终止阶段, 即在关闭过程中, 当连接处于 TIMED WAIT 状态时所使用的计时器。它的超时值为两倍于最大数据包生存期, 主要用来确保当连接被关闭后, 该连接上创建的所有数据包都已完全消失。

6.5.10 TCP 拥塞控制

我们把 TCP 的关键功能之一保留到最后, 即拥塞控制。当提供给任何网络的负载超过它的处理能力时, 拥塞便会产生。Internet 也不例外。当路由器上的队列增长到很大时, 网络层检测到拥塞, 并试图通过丢弃数据包来管理拥塞。传输层接收到从网络层反馈来的拥

塞信息，并减慢它发送到网络的流量速率。在 Internet 上，TCP 在控制拥塞以及可靠传输中发挥着主要的作用。这就是为什么 TCP 是如此特殊的一个协议。

我们在 6.3 节中介绍了拥塞控制的一般情况。关键在于如果一个传输协议使用 AIMD（加法递增乘法递减）控制法则来响应从网络传来的二进制拥塞信号，那么该传输协议应该收敛到一个公平且有效的带宽分配。TCP 的拥塞控制就是在这个方法的基础上实现的，它使用了一个窗口，并且把丢包当作二进制信号。要做到这一点，TCP 维持一个拥塞窗口（congestion window），窗口大小是任何时候发送端可以往网络发送的字节数。相应的速率则是窗口大小除以连接的往返时间。TCP 根据 AIMD 规则来调整该窗口的大小。

回想一下，除了维护一个拥塞窗口外，还有一个流量控制窗口，该窗口指出了接收端可以缓冲的字节数。要并发跟踪这两个窗口，可能发送的字节数是两个窗口中较小的那个。因此，有效窗口是发送端认为的应该大小和接收端认为的应该大小两者中的较小者。真所谓一个巴掌拍不响。如果拥塞窗口或流量控制窗口暂时已满，则 TCP 将停止发送数据。如果接收端说“发送 64 KB 数据”，但发送端知道超过 32 KB 的突发将阻塞网络，它就只发送 32KB；另一方面，如果接收端说“发送 64 KB 数据”，发送端知道高达 128 KB 的突发通过网络都毫不费力，它会发送要求的全部 64 KB。流量控制窗口在前面已经所述过，下面我们只描述拥塞窗口。

现代拥塞控制被添加到 TCP 的实现中很大程度上要归功于（Van Jacobson, 1988）的努力。这是一个引人入胜的故事。从 1986 年开始，早期 Internet 的日益普及导致了第一次出现称为拥塞崩溃（congestion collapse）的问题，由于网络拥塞，在很长时间内实际吞吐量急剧下降（即下降幅度超过了 100 倍）。Jacobson（和许多其他人）着手了解发生了什么事并试图采取补救措施。

Jacobson 实施的高层次修补程序近似于一个 AIMD 拥塞窗口。TCP 拥塞控制中最有趣也是最复杂的部分在于如何在不改变任何消息格式的前提下，把这个修补措施部署到现有的 TCP 实现中。开始，他观察到丢包可以作为一个合适的拥塞信号。这个信号来得有点晚（因为网络早已拥挤不堪），但它相当值得信赖。毕竟，很难建造一个超负荷时也不丢包的路由器，这是一个不太可能改变的事实。即使可用兆兆位（terabyte）的存储器来缓冲大量的数据包，我们或许会有兆位/秒的网络，它们依然会填满这些存储器。

然而，用丢包作为拥塞信号依赖于传输错误相对比较少见的场合。对于诸如 802.11 的无线链路，传输错误比较常见，这也就是为什么它们在链路层有自己的重传机制。由于无线重传，网络层因传输错误导致的丢包通常被屏蔽在无线网络中。这也是在其他链路上很罕见的情况，因为电线和光纤通常具有较低的误码率。

Internet 上的所有 TCP 算法都假设丢包是由拥塞和监控器超时所引起的，而且寻找引起麻烦的蛛丝马迹犹如矿工观看他们饲养的金丝雀一样困难。需要一个良好的重传计时器来准确检测到丢包信号，而且必须以一种及时的方式。我们已经讨论了 TCP 重传计时器如何包括了往返时间的均值和方差的估算。为了修复该计时器，Jacobson 工作中很重要的一步是在估算中要包括方差因子。有了良好的重传超时，TCP 发送端可以跟踪发出去的字节，这些就是网络的负载。它只是简单地着眼于传输的序号和确认的序号之间的差异。

现在看来我们的任务似乎很容易。我们所需要做的是使用序号和确认号来跟踪拥塞窗口，并使用一个 AIMD 规则来调整拥塞窗口的大小。正如你可能已经预料到的，这项工作



做起来比听上去的更复杂。首先要考虑的是数据包发送到网络的方式，即使在很短的时间内也必须匹配网络路径，否则就有可能造成拥塞。例如，考虑一个主机有一个 64 KB 的拥塞窗口，它通过一条 1 Gbps 的链路连到交换式以太网。如果主机一次发送整个窗口，这样的突发流量或许要穿过前方路径上的某条缓慢的 1 Mbps ADSL 线路。这个突发在 1 Gbps 线路上只需要半毫秒，但在 1 Mbps 的线路上则需要半秒，因而阻塞了该条线路，完全扰乱了诸如 IP 语音这样的传输协议。这种行为告诉我们，设计一个能避免拥塞的协议而不是一个能控制拥塞的协议或许是更好的主意。

然而，事实证明，我们可以使用小的突发数据包来突出我们的优势。图 6-43 显示了一个突发数据包的过程。在这个例子中，快速网络上的发送端（1 Gbps 链路）突发 4 个包给一个慢速网络上的接收端（1 Mbps 链路），这条链路是路径上的瓶颈或者最慢的部分，我们来看会发生什么情况。最初的 4 个包以发送端的发送速率尽快通过链路；因为通过一条慢速链路发送数据包所需要的时间比路由器从快速链路上接收下一个数据包所需要的时间更长，所以在路由器中，它们有的正在被发送，有的在排队等候输出。但是队列并不长，因为只有少量的数据包被一次发送过来。特别注意在慢速链路上数据包的长度有所增长。同样的数据包，比如说 1 KB，在慢速链路上的发送时间比在快速链路上的发送时间更长，因而相对链路而言，数据包的长度更长。

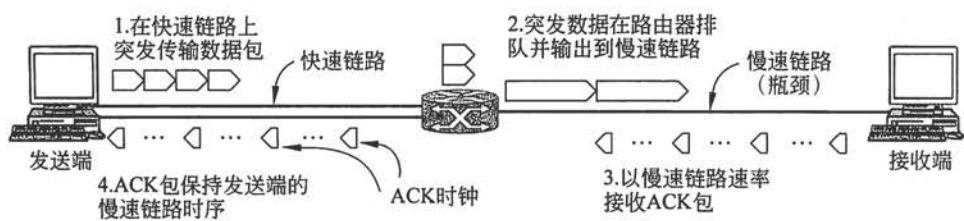


图 6-43 发送端的突发数据包以及返回的确认时钟

最终，数据包抵达接收端，在那里它们得到确认。确认的时间反映了数据包穿越慢速链路后到达接收端的时间。相比在快速链路上的原始数据包，这些确认更分散。随着这些确认穿过网络并传回到发送端，它们保留了这个时序。

关键的观察是这样的：确认返回到发送端的速率恰好是数据包通过路径上最慢链路时的速率。这正是发送端应该使用的精确发送速率。如果发送端以这个速率往网络注入新的数据包，这些数据包就能以慢速链路允许的速率一样快的速度被转发出去，但它们不会再排队和堵塞沿途上的任何一个路由器。这个时序就是确认时钟（ack clock）。这是 TCP 的基本组成部分。通过使用一个确认时钟，TCP 平滑输出流量和避免不必要的路由器队列。

第二个考虑是如果网络拥塞窗口从一个很小的规模开始，那么在快速网络上应用 AIMD 规则将需要很长的时间才能达到一个良好的操作点。考虑一个适度的网络路径，它可以支持 10 Mbps 的传输速率，并且往返时间 RTT 为 100 毫秒。适当的拥塞窗口是带宽延迟积，即 1 MB 或 100 个具有 1250 字节的数据包。如果拥塞窗口从 1 开始，每个 RTT 递增 1 个数据包，那么需要 100 个 RTT，或 10 秒，连接才能以正确的速率运行。在达到正确的传输速度之前要等待一段漫长的时间。如果从一个更大的初始窗口开始，比如 50 个数据包，那么我们就可以减少启动时间。但是，这个窗口对于慢速或者短程链路又太大了点，如果一次全部用完窗口，它会造成拥堵，正如我们刚才所讨论的那样。

相反,针对这两种情况 Jacobson 选择的解决方案是一个线性增长和乘法增长相混合的方法。当建立连接时,发送端用一个很小的值初始化拥塞窗口,最多不超过 4 个段;在 RFC 3390 中描述了该方案的细节,早期的实现采用的初始值是 1,后根据经验改成了用 4 作为初始值。然后发送端发送该初始窗口大小的数据。数据包必须经过一个往返时间才被确认。对于每个重传计时器超时前得到承认的段,发送端的拥塞窗口增加一个段的字节量。此外,随着该段获得确认,现在网络中又少了一段。结果是每一个被确认的段允许发送两个段,每经过一个往返时间拥塞窗口增加一倍。

这种算法称为慢速启动 (slow start),但它根本不慢——它呈指数增长——相比以前的算法,即一次发送整个流量控制窗口大小的数据,它应该算启动得不快。慢速启动算法如图 6-44 所示。在第一次往返时间,发送端把一个数据包注入网络(并且接收端接收到一个数据包)。在接下来的一个往返时间,发送端发出两个数据包,然后在第三个往返时间发送四个。

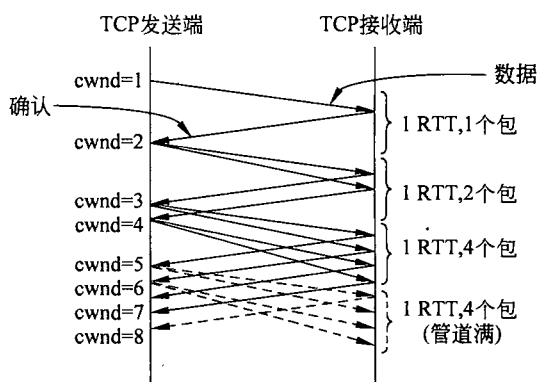


图 6-44 初始拥塞窗口为一个段的慢速启动过程

慢速启动在一定范围内的链接速度和往返时间上都能行之有效,并且使用确认时钟将发送端的传输速率与网络路径相匹配。考查图 6-44 中确认段从其发送端发送到接收端的方式。当发送端得到一个确认,它就吧拥塞窗口大小增加 1,并立即将两个数据包发送到网络中(其中一个包是新增加的一个;另一个包是替代那个已被确认并离开网络的数据包。任何时候未确认的数据包的数量就是拥塞窗口的大小)。不过,这两个数据包到达接收端的紧随程度不一定非要和它们被发送出来的间隔一样。例如,假设发送端在 100 Mbps 的以太网上。每个 1250 字节长的数据包需要 100 微秒的发送时间。因此两个数据包之间的延迟只有 100 微秒。但是如果这些数据包在当前路径的任何地方穿过一条 1 Mbps 的 ADSL 链路时,形势就发生了变化。现在发送同样的数据报需要 10 毫秒。这意味着两个数据包之间的最小时间间隔增长了 100 倍。除非数据包必须在此后的某条链路上排队等候,否则它们之间的间距仍然很大。

在图 6-44 中,这种效果体现在强制数据包以它们之间的最小间距到达接收端。接收端在发送确认时要维持这种间隔,从而发送端也会以同样的间隔接收确认。如果网络路径缓慢,确认将抵达得较晚(超过一个 RTT 的延迟)。如果网络路径很快,确认会迅速到达(同样,在一个 RTT 后)。发送端所要做的只是按照确认时钟的时间来注入新的数据包,这就是慢速启动。

由于慢速启动导致拥塞窗口按指数增长，最终（很快而不是很晚）它将太多的数据包以太快的速度发到网络。当发生这种情况时，网络中将很快建立起队列。当队列满时，一个或多个包会被丢弃。此后，当确认未能如期返回发送端时，TCP 发送端将超时。有证据表明，如图 6-44 的慢速启动增长太快。经过 3 个 RTT，网络中有四个数据包。这四个数据包经过完整的 RTT 后到达接收端。也就是说，四个包的拥塞窗口大小就是该连接的正确大小。然而，随着这些数据包被确认，慢速启动持续增长拥塞窗口，在另一个 RTT 达到 8 个包。这些数据包中只有四个在一个 RTT 内到达接收端，无论发送了多少个数据包。也就是说，网络管道已经为满。发送端把额外的数据包放置到网络中将在路由器上产生队列，因为它们被传递到接收器的速度不够快。拥塞和丢包即将发生。

为了保持对慢速启动的控制，发送端为每个连接维持一个称为慢启动阈值（slow start threshold）的阈值。最初，这个值被设置得任意高，可以达到流量控制窗口的大小，因此它不会限制连接速度。TCP 以慢速启动方式不断增加拥塞窗口，直到发生超时，或者拥塞窗口超过该阈值（或接收端的窗口为满）。

每当检测到丢包，比如超时了，慢启动阈值就被设置为当前拥塞窗口的一半，整个过程再重新启动。基本想法是当前的窗口太大，因为是它在过去造成了阻塞，所以现在才检测到超时。一较早时间成功使用的一半窗口也许是拥塞窗口的更好估计值，它更接近路径的容量而不会造成丢失。我们在图 6-44 的例子中，当拥塞窗口增长到 8 个数据包时，可能造成丢失，而在前一个 RTT 中具有 4 个数据包的拥塞窗口就是正确值。然后，拥塞窗口被复位到其初始值，恢复慢速启动。

一旦慢速启动超过了阈值，TCP 就从慢速启动切换到线性增加（即加法递增）。在这种模式下，每个往返时间拥塞窗口只增加一段。像慢速启动一样，这通常也是为每一个被确认的段而不是为每一次 RTT 实施窗口的增加。回忆一下，拥塞窗口 $cwnd$ 和最大段长 MSS ，一个常见的近似做法是这样的：针对 $cwnd/MSS$ 中可能被确认的每个数据包，将 $cwnd$ 增加 $(MSS \times MSS) / cwnd$ 。这种增长速度并不需要很快。整个想法是在一个 TCP 连接所花费的大量时间中，它的拥塞窗口接近最佳值——不至于小到降低吞吐量，但也没大到发生拥塞情况。

线性递增如图 6-45 所示，这是相同情况下的慢速启动。在每一个 RTT 结尾，发送端的拥塞窗口增长到足够它可以向网络注入额外一个数据包。相比慢速启动，线性增长速率要慢得多。这对于小的拥塞窗口差别并不大，就像这里的例子一样，但当随着时间拥塞窗口增长到一定程度时，比如说 100 段，差异就大了。

我们还可以做些其他事情来提高性能。到目前为止这个方案中的缺陷是等待超时。超时时间相对较长，因为它们必须是保守的。当一个包被丢失后，接收端不能越过它确认，因此确认号将保持不变，发送端因为拥塞窗口为满将无法发送任何新包到网络。这种情况可以持续一段比较长的时期，直到计时器被触发和重传丢失的包。在这个阶段，TCP 再次慢速启动。

发送端有一个快速方法来识别它的包已经被丢失。当丢失数据包的后续数据包到达接收端时，它们触发给发送端返回确认。这些确认段携带着相同的确认号，称为重复确认（duplicate acknowledgement）。发送端每次收到重复确认时，很可能另一个包已经到达接收端，而丢失的那个包仍然没有出现。

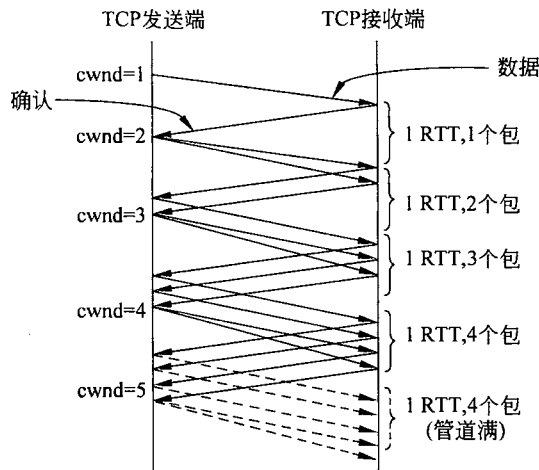


图 6-45 初始拥塞窗口为一个段的线性增长过程

因为包可以选择网络中不同的路径，所以它们可能不按发送顺序到达接收端。这样就会触发重复确认，即使没有数据包被丢失。然而，这种情况在 Internet 上并不多见。当存在跨越多条路径的包时，收到的数据包通常需要重新排序的不会太多。因此，TCP 有点随意地假设三个重复确认意味着已经丢失一个包。丢失包的序号可以从确认号推断出来，它是整个数据序列中紧接着的下一个数据包。因此，这个包可以被立即重传，在其计时器超时前就重新发送出去。

这种启发式机制称为快速重传 (fast retransmission)。重传后，慢启动阈值被设置为当前拥塞窗口的一半，就像发生了超时一样。重新开始慢启动过程，拥塞窗口被设置成一个包。有了这个窗口大小，如果在一个往返时间内确认了该重传的数据包以及丢包之前已发送的所有数据，则发出一个新的数据包。

图 6-46 给出了迄今为止我们已经构建的拥塞算法示意图。此版本的 TCP 称为 TCP Tahoe，它包括了 1988 年发布的 4.2BSD TCP Tahoe。这里的最大段长为 1 KB。最初，拥塞窗口为 64 KB；但发生超时后，这样的阈值被设置为 32 KB，并且拥塞窗口被设置为 1 KB，从 0 开始传输。拥塞窗口呈指数增长，直到达到阈值 (32 KB)。每次到达一个新的确认就增加窗口的大小，而不是连续增加，因而导致窗口的大小表现出离散的阶梯模式。当窗口大小达到阈值后，窗口大小就按线性增长。每个 RTT 只增加一段。

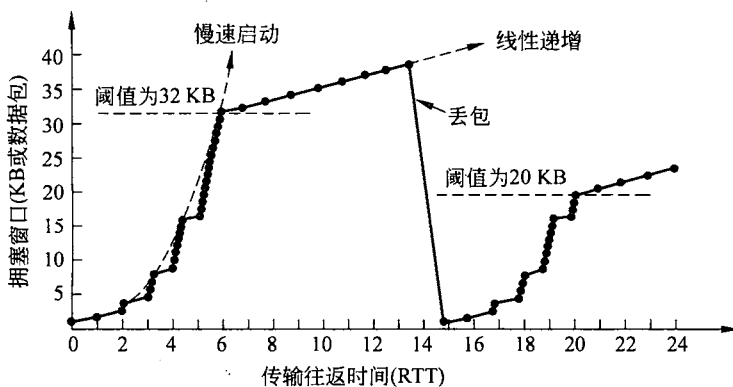


图 6-46 TCP Tahoe 的慢速启动后面接着线性递增

第 13 轮传输很不幸（它们应该都知道），其中一个包在网络中丢失了。当到达了 3 个重复确认时，发送端就检测出丢包行为。此时，发送端重传丢失的包，并将阈值设置为当前窗口的一半（现在是 40 KB，因此一半是 20 KB），并再次开始慢速启动算法。重新启动时，拥塞窗口大小为一个数据包，在往返时间内，所有以前传输的数据要离开网络并得到承认，而且重传数据包也要得到确认。拥塞窗口的增长与以前慢速启动时一样，直到它达到 20 KB 的新阈值；然后，窗口再次以线性模型增长。窗口持续以线性增长直到通过重复确认检测到另一个丢包或超时（或者接收端的窗口达到上限）。

TCP Tahoe（其中包括良好的重传计时器）提供了一个可实际工作的拥塞控制算法，它解决了网络的拥塞崩溃问题。Jacobson 意识到它有可能做得更好。在快速重传时，连接还工作在一个太大的拥塞窗口，但它仍然同样以确认时钟的速率在运行。每次到达另一个重复确认时，可能另一个数据包已经离开网络。使用重复确认来计算网络中的数据包，有可能让一些包离开网络，并继续为每一个额外的重复确认发送一个新的包。

快速恢复（Fast recovery）就是实现这种行为的启发式机制。这是一个临时模式，其目的是保持拥塞窗口上运行确认时钟，该拥塞窗口有一个新阈值或者快速重传时把拥塞窗口值减半。要做到这一点，对重复确认要计数（包括触发快速重传机制的那三个重复确认），直到网络内的数据包数量下降到新阈值。这大概需要半个往返时间。从此时往后，每接收到一个重复确认就发送一个新的数据包。快速重传之后的一个往返时间后，丢失的包将被确认。在这个时间点，重复确认流将停止，快速恢复模式就此退出。拥塞窗口将被设置到新的慢启动阈值，并开始按线性增长。

这种启发式的结果是 TCP 避免了慢速启动，只有第一次启动或者发生超时时才进入真正的慢速启动。当发生多个数据包丢失时，快速重传机制不足以恢复，仍然可能会发生超时。此时，不是反复地进入慢速启动，而是当前连接的拥塞窗口遵循一种锯齿（sawtooth）模式，即加法递增（每个 RTT 增加一段）和乘法递减（每个 RTT 减半）。这正是我们力求实现的 AIMD 规则。

这种锯齿行为如图 6-47 所示。它由 TCP Reno 提供，这是以 1990 年发布的 4.3BSD Reno 命名的，在那个早期版本中包括了上述机制。TCP Reno 基本上是 TCP Tahoe 加上快速恢复机制。经过一个初始的慢速启动，拥塞窗口大小直线攀升，直至通过重复确认检测到丢包。丢失的数据包被重传，并且启用快速恢复机制来维持确认时钟运行，直到重传的数据包被

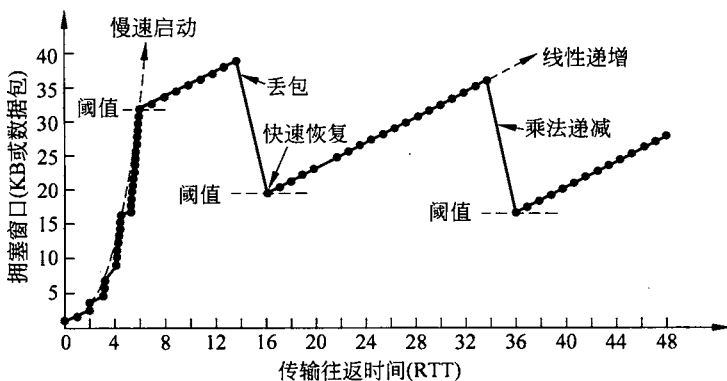


图 6-47 TCP Reno 的快速恢复和锯齿模式

确认。此时，拥塞窗口恢复到新的慢启动阈值，而不是从 1 开始。这种行为无限期地继续下去，在连接花费的大部分时间内其拥塞窗口接近于带宽延迟积的最佳值。

TCP Reno 及它用来调整拥塞窗口的机制形成了 TCP 的拥塞控制，至今已有超过二十年的历史。在这期间的大多数变化只是以轻微的方式调整这些机制，例如，修改了初始窗口的选择和消除了各种含糊不清的定义。一些改进针对从数据包窗口中两个或两个以上数据包的丢失中恢复过来。例如，TCP NewReno 版本在重传后使用部分提前的确认号来发现并修复另一个丢失的包 (Hoe, 1996)，RFC 3782 对此进行了详细的描述。自 20 世纪 90 年代中期以来，出现了一些 TCP 变种，所有这些变种都遵循我们描述过的原则，但使用了略微不同的控制法则。例如，Linux 使用的 TCP 变种称为 CUBIC TCP (Ha 等, 2008)，Windows 则包括了一个称为组合 TCP (Compound TCP) 的变种 (Tan 等, 2006)。

两个较大的变化对 TCP 的实现也有影响。首先，TCP 的许多复杂性来自于从一个重复确认流中推断出已经到达和已经丢失的数据包。累计确认号无法提供这种确切的信息。一个简单的解决方法是使用选择确认 (SACK, Selective ACKnowledgement)，该确认列出了 3 个已接收的字节范围。有了这个信息，发送端在实现拥塞窗口时可以更直接地确定哪些数据包需要重传，并跟踪那些还在途中的数据包。

当发送端和接收端建立连接时，它们各自在 TCP 段的选项字段发送允许 SACK，通知对方它们理解选择确认。一旦在连接上启用 SACK，它的工作原理如图 6-48 所示。接收端在正常的方式下使用 TCP 的确认号字段，作为已收到的最高顺序字节的累积确认。当它接收到乱序的数据包 3 时（因为数据包 2 丢失了），它发送一个确认段，其中包括针对包 1 的（重复）累计确认和包 3 的 SACK 选项。SACK 选项给出高于累计确认已收到的字节范围。第一个范围是引发重复确认的数据包；接下来的范围，如果存在的话，是旧的数据块。通常采用三个范围。当收到包 6 时，要用到两个 SACK 的字节范围，来指明包 6 和包 3~包 4 已收到，还有额外的到包 1 的所有数据包。从它接收到的每个 SACK 选项中包含的信息，发送端可以决定重发哪些数据包。在这种情况下，重传数据包 2 和包 5 将是一个好主意。

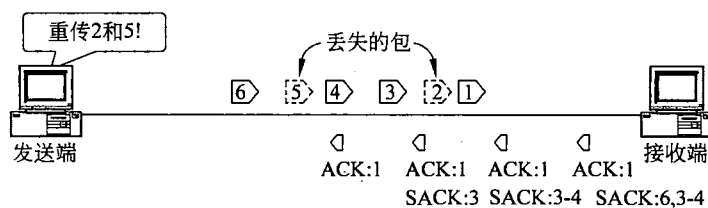


图 6-48 选择确认

SACK 是严格意义上的咨询信息。实际上使用重复确认来检测丢包和调整拥塞窗口还是像以前一样。然而，有了 SACK，TCP 可以更加容易地从同时丢失多个包的情况下恢复过来，因为 TCP 发送端知道哪些数据包尚未收到。SACK 目前已被广泛部署。RFC 2883 对 SACK 做了描述，而关于 TCP 拥塞控制如何使用 SACK 则在 RFC 3517 中描述。

第二个变化是除了用丢包作为拥塞信号外，显式拥塞通知 (ECN, Explicit Congestion Notification) 的使用。ECN 是 IP 层的机制，主要用来通知主机发生了拥塞，我们在 5.3.4 节中有过描述。有了它，TCP 接收端可以接收来自 IP 的拥塞信号。

当发送端和接收端在建立阶段过程中设置了 ECE 和 CWR 标志位，双方均表示它们能

够使用这些标志位后，该 TCP 连接就可启用 ECN。如果使用 ECN，每个携带 TCP 段的数据包在 IP 头打上标记，表明它可以携带 ECN 信号。支持 ECN 的路由器在接近拥塞时就会在携带 ECN 标志的数据包上设置拥塞信号，而不是在拥塞发生后丢弃这些数据包。

如果到达的任何数据包携带了 ECN 拥塞信号，则会告知 TCP 接收端。然后接收端使用 ECE (ECN Echo) 标志位给 TCP 发送端发通知，告知它的数据包经历了拥塞。发送端通过拥塞窗口减少 (CWR, Congestion Window Reduced) 标志位告诉接收端它已经收到拥塞信号了。

TCP 发送端收到这些拥塞通知消息的反应和它根据重复确认检测到丢包的处理方式完全相同。不过，严格来说这种情况更好。拥塞已经被检测出来，而且没有包受到任何方式的损害。ECN 的详细描述见 RFC 3168。它要求主机和路由器都必须支持该功能，目前尚未在 Internet 上广泛应用。

有关 TCP 实现中拥塞控制行为的详细信息，请参见 RFC 5681。

6.5.11 TCP 未来

作为 Internet 的主力军，TCP 已经被用于许多应用，并随着时间的推移做了许多扩展以便在广泛的网络上性能表现良好。许多实际部署的版本在实现上跟我们所描述的经典算法略微有所不同，尤其是拥塞控制和针对攻击的鲁棒性方面。很有可能 TCP 将伴随着 Internet 的发展而持续地改进着。这里，我们会特别提到两个问题。

第一个问题是 TCP 没有为所有应用程序提供所需的传输语义。例如，某些应用程序需要保留其发送的消息或记录边界。其他的应用程序要处理一组相关的对话，如 Web 浏览器从同一台服务器传输几个对象。还有其他应用程序希望更好地控制它们使用的网络路径。但是 TCP 的标准套接字接口并不满足这些需求。从本质上讲，应用程序承担着解决任何 TCP 没有解决的问题的负担。这导致了研究者们提出了具有稍微不同接口形式的新协议的提议，其中两个比较著名的例子是由 RFC 4960 定义的流控制传输协议 (SCTP, Stream Control Transmission Protocol) 和结构化流传输 (SST, Structured Stream Transport) (Ford, 2007)。然而，每当有人提出要对一些已经很好地工作了很长时间的東西改革时，总会爆发“用户要求更多的功能”和“如果不打破就不能解决”两大阵营之间的巨大战争。

第二个问题是拥塞控制。你或许期望经过我们的审议以及随着时间推移而开发出来的一系列机制已经解决了这个问题。事实并非如此。我们介绍的 TCP 拥塞控制的形式虽然已经被广泛使用，但这是基于丢包作为拥塞的信号。当 (Padhye 等, 1998) 根据锯齿模式为 TCP 吞吐量建立模型时，他们发现随着速度的加快丢包率呈现急剧下降的趋势。对于往返时间为 100 毫秒，数据包长度为 1500 字节，要达到 1 Gbps 的吞吐量，大约每隔 10 分钟要丢一个包。这种情况下的丢包率大约是 2×10^{-8} ，显然这个数字令人难以置信的小。把丢包率作为一个良好拥塞信号实在是太罕见了，任何其他来源的损失（例如，数据包的传输误码率 10^{-7} ）可以很容易地发挥主宰作用，由此真正限制了吞吐量。

这种关系在过去不是一个问题，但网络速度越来越快，导致许多人重写拥塞控制。一种可能性是干脆使用另一种拥塞控制，不把丢包当作拥塞信号。我们在 6.2 节中给出了几个例子。拥塞信号可以是往返时间，当网络变得拥塞时往返时间会增长，如 FAST TCP 所

用的那样 (Wei 等, 2006)。其他方法也是可能的, 时间会告诉我们什么是最好的控制方式。

6.6 性能问题

在计算机网络中, 性能问题非常重要。当成百上千台计算机相互连接在一起时, 无法预知结果的复杂交互过程很常见。这种复杂性常常会导致很差的性能, 而且无人知道其中的缘由。在下面的章节中, 我们将讨论许多与网络性能有关的问题, 以便了解可能存在哪些问题以及如何处理这些问题。

不幸的是, 理解网络性能更像是一门艺术, 而不是一门科学。这里很少有可在实践中应用的基础理论。我们能够做的最好方法是给出一些来自于实践的经验规则, 并且展示一些真实世界中的例子。我们有意将这部分讨论推迟到学习了 TCP 的传输层之后, 因为应用程序获得的性能依赖于传输层、网络层和链路层的结合, 而且能够在不同场合中使用 TCP 作为例子。

在下面的章节中, 我们将考查网络性能的如下 6 个方面。

- (1) 性能问题。
- (2) 网络性能的测量。
- (3) 快速网络中的主机设计。
- (4) 快速处理段。
- (5) 头的压缩。
- (6) “长肥”网络的协议。

上述这些方面同时从主机和网络, 以及网络速度和规模增长来考虑网络性能。

6.6.1 计算机网络中的性能问题

有些性能问题 (比如拥塞) 是由于临时资源过载所引起的。如果到达一台路由器的流量突然超过了它的处理能力, 那么, 拥塞就会发生, 从而导致性能问题。在本章和前一章中我们已经详细地学习了拥塞控制。

当网络中存在结构性的资源不平衡时, 性能也会退化。例如, 如果将一条千兆位的通信线路连接到一台低档的 PC 上, 则性能较差的 CPU 将无法快速地处理入境数据包, 因此有些数据包将会被丢失。这些数据包最终会被重传, 既增加了延迟, 又浪费了带宽, 而且往往降低了性能。

过载也可能被同步触发。例如, 如果一个段包含了一个坏参数 (比如, 它的目标端口), 那么, 在许多情形下, 接收端会尽职地返回一个错误通知。现在请考虑, 如果将一个坏段广播给 1000 台机器将发生什么样的情况: 每台机器都可能送回一个错误消息。由此引发的广播风暴 (broadcast storm) 可能会严重地削弱网络的性能。以前, UDP 协议就遭受过这个问题, 直到 ICMP 协议作了修改, 使主机不再对发送给广播地址的 UDP 段中错误做出响应, 以免遭受广播风暴。对于无线网络来说, 由于广播本质上会发生, 而且无线带宽有限, 因此尤其要小心避免未经检查的广播响应。

同步触发过载的另一个例子是掉电之后发生的情形。当电源恢复时，所有的机器同时启动系统。一个典型的启动序列可能如下：首先与某一台（DHCP）服务器联系，以便获得一个真实的身份；然后与某一台文件服务器联系，以便获得操作系统的一份副本。如果一个数据中心的成千上万台机器同时做这些事情，则服务器可能因不堪重负而崩溃。

即使不存在同步过载，并且资源也足够，同样也可能由于缺少系统总体协调而发生性能退化的现象。例如，如果一台机器有足够的 CPU 能力和内存，但是并没有为缓冲区空间分配足够多的内存，则流量控制机制将放缓段的接收，从而限制了传输性能。当 Internet 变得越来越快，而流量控制窗口仍然维持在 64 KB 大小时，许多 TCP 连接都存在问题。

另一个调整问题是超时间隔的设置。当一个段被发送出去时，TCP 通常会设置一个计时器来预防该段的丢失。如果超时间隔设置得太短，将会发生不必要的重传，从而堵塞线路；如果超时间隔设置得太长，当段被丢失之后将导致不必要的延迟。其他可以调整的参数包括捎带确认应该等待数据段多长时间之后未果才发送单独的确认以及应该尝试经过多少次重传之后再放弃。

对于诸如音频和视频的实时应用，它们的另一个性能问题是抖动。仅仅有足够的平均带宽还不足以有良好的性能。这些应用还要求较小的传输延迟。要想同时获得较短的延迟必须小心规划网络中的流量负载，同时还需要链路层和网络层共同支持服务质量。

6.6.2 网络性能测量

当一个网络的运行效果很差时，它的用户通常会向网络运营商抱怨并要求提高网络质量。为了改善网络性能，网络操作人员首先必须弄清楚发生了什么问题。为了找出真正的问题所在，操作人员需要对网络进行测量。在本小节中，我们来考查网络性能的测量问题。下面的许多讨论以（Mogul, 1993）的工作为基础。

测量工作可以有許多不同的方式，也可以在不同的地点进行（既指物理位置，也指协议栈中的位置）。最基本的一种测量手段是在开始某一个动作时启动一个计时器，然后确定该动作需要多长时间。例如，知道一个段需要多长时间才能被确认是很关键的一个测量指标。其他一些测量指标可以通过计数器来完成，即记录某种事件发生的次数（比如丢失的段数量）。最后，人们通常对于某些事物的数量比较感兴趣，比如在特定的时间间隔内所处理的字节数。

测量网络的性能和参数有许多潜在的陷阱。以下我们列出其中一部分。任何一种系统化的网络性能测量手段都应该小心地避免这些陷阱。

确保样值空间足够大

不要仅仅测量发送一个段的时间，而是要重复地测量，比如说测量 100 万次，然后再取平均值。启动效应可能放慢第一个段的速度，而且排队会引入变化，比如 802.16 NIC 或者线缆调制解调器在一个空闲周期后获得带宽预留。采用大量的样本可以减小测量均值和标准方差中的不确定性。这种不确定性可以利用标准的统计公式计算出来。

确保样值具有代表性

理想情况下，这一百万次测量的完整序列应该分布在一天或者一周的不同时刻重复进行，以此来看不同的网络条件对测量数值的影响。例如，对于拥塞的测量，如果仅仅在没有拥塞的那一刻来测量网络拥塞，那么这样的测量以及结果没什么用。有时候测量结果初看起来可能不符合直觉，比如在上午 11 点和下午 1 点钟网络严重拥塞，但是中午时候却没有拥塞（所有的用户都去吃午饭了）。

在无线网络中，信号的传播位置是一个很重要的变量。由于天线的差异，即使测量节点放置在靠近无线客户端也可能无法观察到与客户端相同的数据包。在研究中最好从无线客户端进行测量，考查它究竟看到了什么。如果做不到这一点，那就使用技术把从不同角度的无线测量结合起来以便对正在发生的事情有个更全面的了解（Mahajan 等，2006）。

缓存可以破坏测量结果

如果协议使用了缓存机制，那么重复多次测量将返回一个不可预测的快速答复。例如，获取一个 Web 页面或者查询一个 DNS 名字（来发现其 IP 地址）可能只有第一次涉及网络交换，以后返回的都是从缓冲区获得的结果，没有真正往网络上发送任何数据包。这样的测量结果本质上是没有什么价值的（除非你要测量的就是缓冲性能）。

缓冲机制也有类似的影响。已经有 TCP/IP 性能测试报告称 UDP 可以获得比网络允许的要好得多的性能。这是怎么发生的呢？调用 UDP 时一旦内核接受了消息并且消息被排入传输队列之后，通常控制权就马上返回给应用程序了。如果主机上有足够的缓冲空间，则执行 1000 次 UDP 调用也不意味着所有的数据都已经被发送出去。大多数数据包仍然在内核中，但是性能测试程序却认为它们都已经被传送出去了。

建议测试时，你要绝对确保网络操作是如何缓存以及缓冲数据的。

确保测试期间不会发生不可知事情

测量时如果有人在你的网络上运行一个视频会议，那么通常得到的测量结果和没有视频会议时的测量结果不同。最好在一个空闲的网络上进行测试，并且根据需要自己创建所需的工作负载。不过这种做法也存在缺陷。尽管你认为在凌晨 3 点钟时不会有人使用网络，但有可能自动备份程序恰好在这个时间点开始将所有的磁盘数据复制到磁带上。或者，其他时区的用户可能来访问你制作精美的 Web 页面，也会导致繁重的流量。

无线网络在这方面更具挑战性，因为它往往无法区分各种干扰源。即使附近没有其他无线网络在发送流量，或许有人在用微波炉爆玉米花，不经意间造成了干扰，从而降低 802.11 性能。基于这些原因，良好的做法是监控整个网络的活动，这样你至少可以认识到何时发生了意想不到的事情。

小心使用粗粒度时钟

计算机时钟的功能是以固定的时间间隔递增某个计数器。例如，一个毫秒计时器每隔 1 毫秒计数器就加 1。使用这样的计时器来测量一个持续时间小于 1 毫秒的事件是有可能的，但要非常小心。当然，有些计算机有更精确的时钟，但总会有时间更短的事件需要测量。请注意，时钟并不总是和返回的时间一样精确。

例如，为了测量建立一个连接所需要的时间，应该在进入传输层代码以及离开传输层

代码时，分别读出时钟值（比如说以毫秒为单位计数）。如果真正的连接建立时间是300微秒，则两次读取的时间差值要么是0，要么是1，显然这两个结果都是错误的。然而，如果重复测量一百万次，把所有测量的总和累加起来，再除以一百万，则平均时间比1微秒还要精确得多。

小心推断结果

假设你利用模拟的网络负载来进行测量，网络负载从0（空闲）到0.4（40%的容量）。例如，在802.11网络上发送IP语音包可以用如图6-49中的数据点和实线表示。如果能推断出随着负载增加而响应时间呈线性增长，如图中点线所示那样，那么结果是很诱人的。然而，许多排队论的研究结果都涉及一个 $1/(1-\rho)$ 因子，其中 ρ 是负载，所以真正的值可能看起来更像是虚线，当负载变高时其上升速度远远高于线性增长。也就是说，竞争的影响在高负载下更为明显。

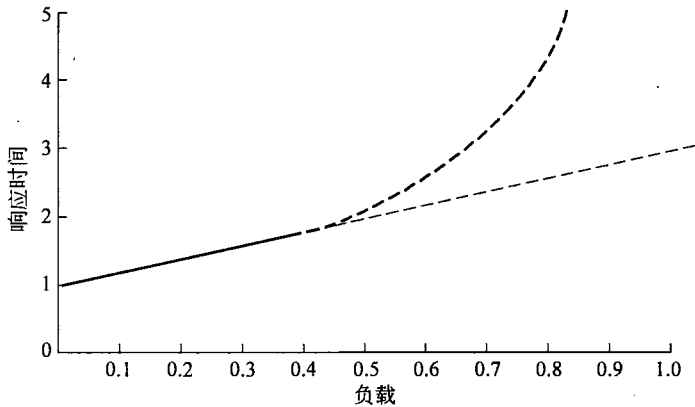


图 6-49 响应时间是负载的函数

6.6.3 针对快速网络的主机设计

测量性能并进行修补可以在一定程度上提高网络性能，但是，它们不可能代替一个从一开始就有的良好设计。一个设计不佳的网络其得以改进的余地毕竟有限，除此之外，它必须从头开始重新设计。

在本节中，我们将就主机实现网络协议提出一些经验法则。令人惊奇的是，经验表明通常的性能瓶颈不在快速网络而在于主机。这里存在两个原因。首先网络接口卡（NIC，Network Interface Cards）和路由器在工程上早已经能以“线速率”（wire speed）运行。这意味着它们处理数据包的速度和数据包到达链路的速度一样快。其次，相关的性能是指应用程序能获得的。它不是链路容量，而是经过网络和传输层处理之后的吞吐量和延迟。

减少软件开销可以提高吞吐量和减少延迟，从而达到提高网络性能的目的。同时，它还能减少花在网络上的能量，能耗这个因素对于移动计算机是很重要的考虑。这些想法中的大多数多年来早已成为大多网络设计师必守的常识。它们首次由（Mogul, 1993）明确提出，我们主要遵循他的观点。另一个相关的知识源来自（Metcalf, 1993）。

主机速度比网络速度更重要

长期的经验表明在几乎所有的高速网络中，操作系统和协议开销占据着线路的主要实际时间。例如，从理论上讲，1 Gbps 以太网上的最小 RPC 时间是 1 微秒，对应着最小（512 字节）的请求和后续的一个最小（512 字节）应答。实际上，克服了软件开销，随时随地使得 RPC 时间接近理论值都是一个了不起的成就，但却很少真正发生。

类似地，在 1 Gbps 上运行的最大问题是应该足够快地将比特从用户缓冲区中取出放到网络上，以及接收主机以比特到达的速度来处理它们。如果你将主机（CPU 和内存）速度加倍，那么，你通常可以获得接近两倍的吞吐量。如果瓶颈在主机，那么加倍网络容量是没有效果的。

减少包技术来降低开销

每个段都有特定数量的开销（例如，头）以及数据（比如有效载荷）。这两部分都需要消耗带宽，同时也都需要处理（例如，处理段头和计算校验和）。发送 100 万字节时，不管段的大小是多大，数据成本是相同的。然而，长度为 128 字节的段意味着每个段的开销是长度为 4 KB 段的开销的 32 倍。带宽和处理开销加起来极大地降低了吞吐量。

每个包在较低层次的开销放大了这种效果。如果主机速度跟得上，每个到达的数据包都会导致一次新的中断。在一个现代化的流水线处理器中，每次中断都要中断 CPU 管道、干扰缓存、需要改变内存管理的上下文、废除分支预测表并强制保存大量的 CPU 寄存器。因此把发送的段减少 n 倍，可以降低 n 倍的中断和数据包开销。

你可能会说，人与计算机执行多任务的能力都很差。这种观察奠定了发送尽可能大 MTU 数据包的需求，这样的 MTU 能通过网络路径而无须分片。诸如 Nagle 算法和 Clark 解决方案的一些机制都试图避免发送小的数据包。

最小化数据预取

实现一个分层协议栈的最简单方法是每一层用一个模块实现。不幸的是，这将导致每一层重复许多工作（或至少通过多次传递才能访问到数据）。例如，在 NIC 收到一个数据包后，它通常是把该包复制到内核缓冲区。从那里，它再被复制到一个网络层的缓冲区，供网络层实体处理；然后该包又被复制到传输层缓冲区，由传输层来处理；最终，由应用程序接收。对于一个入境数据包来说，在将它包含的段递交出去之前被复制三次或四次并不罕见。

所有这些复制都大大降低了协议性能，因为内存操作比寄存器指令慢了一个量级。例如，如果 20% 的指令要对实际内存进行操作（即未命中高速缓存），这在预取入境数据包时是很有可能，则平均指令执行时间要放慢 2.8 倍（ $0.8 \times 1 + 0.2 \times 10$ ）。这里，硬件辅助将无能为力。这个问题的根源在于操作系统执行了太多的复制操作。

一个聪明的操作系统通过把多个层次的处理结合在一起，以此来最小化复制操作。例如，TCP 和 IP 通常实现在一起（正如 TCP/IP 所示的那样），这样当从网络层的处理切换到传输层的处理时，不再需要复制数据包的有效载荷。另一种常见的伎俩是在传递单个数据时在一层内执行多个操作。例如，在复制数据时通常要计算校验和（当必须要复制时），新近计算出的校验和可贴在尾部。

最小化上下文切换

一个相关的规则说明上下文切换（例如，从内核模式到用户模式）是致命的。它们把中断和复制操作结合在一起，而这是个很坏的属性。这个成本就是为什么传输协议通常实现在内核中的理由。就像减少包计数一样，发送数据的库函数可以在内部进行缓冲直到它有大量的数据再发送，通过这种内部缓冲机制可以减少上下文切换。同样，在接收端，小的入境段应收集在一起一举传递给用户而不是个别地传递，以此最大限度地减少上下文切换。

在最好的情形下，一个入境数据包导致从用户模式到内核模式的一次切换，然后再切换到接收进程并将新到达的数据交给它。不幸的是，在许多操作系统中，除此以外还需要额外的上下文切换。例如，如果网络管理器以特殊进程的形式运行在用户空间中，那么，一个数据包的到来可能引发一次从当前用户到内核的上下文切换；然后从内核切换到网络管理器；之后再切换回内核；最后从内核切换到接收进程。这个切换过程如图 6-50 所示。在每个数据包上的所有这些上下文切换对 CPU 时间是个严重浪费，同时也严重地影响了网络性能。

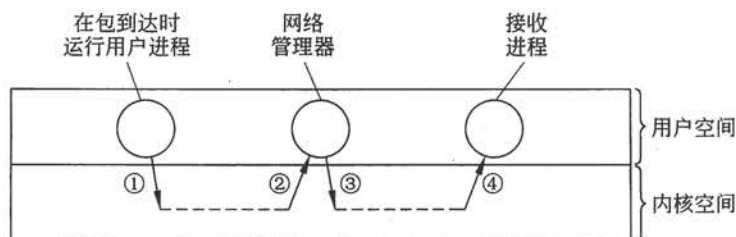


图 6-50 处理一个数据包要经历从用户空间到网络管理器的 4 次上下文切换

避免拥塞比从中恢复更好

“一分预防胜过十分治疗”这句古老的格言无疑特别适用于网络拥塞。当网络拥塞时，数据包被丢失，带宽被浪费，还引入了无用的延迟等一系列问题。所有为此付出的成本都是没有必要的，而且从拥塞中恢复还需要时间和耐心。最好的做法是从一开始就不让拥塞发生。避免拥塞就好像你接种 DTP 疫苗：当你接种的时候你会感觉疼一下，但在将来使你避免更多的疼痛。

避免超时

计时器在网络中是必要的，但应该尽量少用计时器，而且应该尽量少发生超时。当一个计时器超时，通常需要重复执行某个动作。如果确实需要重复执行这个动作，则执行该动作；否则，不必要的重复就是一种浪费。

避免额外工作的办法是小心设置计时器的超时值，将计时器设置成稍稍超过一点保守时间边界。如果将计时器的超时间隔设置得太长，那么所在的连接（不太可能）出现段丢失事件时会增加少量的额外延迟。如果一个计时器过早超时，则会消耗掉主机资源、浪费带宽，而且毫无理由地给很多路由器增加额外的负载。

6.6.4 快速处理段

既然我们已经覆盖了通用规则，我们将考查某些加快段处理速度的特殊方法。要想了

解更多的信息，请参考（Clark 等，1989；Chase 等，2001）。

段的处理开销由两部分组成：每个段的处理开销和每个字节的处理开销。这两部分都有改进的余地。快速处理段的关键是分离出一般情况和成功情况（单向数据传输），并对它们作特殊处理。许多协议倾向于强调出现某种错误情况时该怎么处理（比如，丢失了一个数据包），但是若要使得协议快速运行，协议的设计者应该把目标定在一切正常运行时如何最小化处理时间，而把出现问题时如何最小化处理放在次要地位考虑。

尽管为了进入 ESTABLISHED 状态需要一系列特殊的段，但一旦进入该状态，段的处理就是直截了当的，直到连接的某一端关闭连接。让我们从 ESTABLISHED 状态下的发送端开始，看看它有数据要发送时的情形。为了清楚起见，假设传输实体运行在内核中，尽管同样的想法适用于其他的情形：传输实体是一个用户空间的进程，或者是发送进程内部的一个库。在图 6-51 中，发送进程陷入内核并执行 SEND。传输实体要做的第一件事情是测试，检查是否为正常情形：即状态必须为 ESTABLISHED，两端均不打算关闭当前连接，发送的是一个普通段（即不是带外数据），而且接收端有足够的窗口空间可供使用。如果上述所有的条件都满足，则无须进一步测试，使用发送端传输实体的快速路径。在通常情况下，这条快速路径占据了大部分的处理时间。

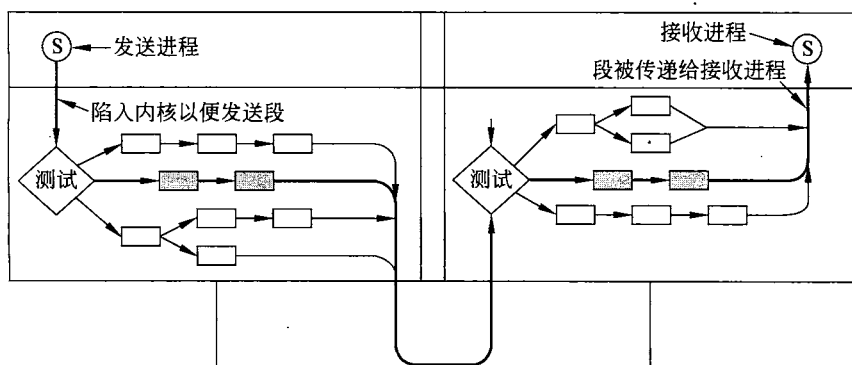


图 6-51 一条重载线路上的快速路径（阴影标示了这条路径上的处理步骤）

通常情况下，连续的数据段的头几乎是相同的。为了充分利用这个事实，传输实体在内部保存一个原型头。在快速路径的起始处，该原型头被尽可能快地逐字复制到一个临时缓冲区中。对于那些段与段之间不同的字段，则复制时覆盖掉缓冲区中的相应字段。这些字段往往可以很容易地从状态变量中导出，比如“下一个序号”。然后，一个指向完整段头的指针和一个指向用户数据的指针被传递给网络层；因此，网络层可以遵循同样的策略（图 6-51 没有显示）。最后，网络层将结果数据包交给数据链路层传输。

上述原理在实际中是如何工作的，我们用一个 TCP/IP 的一个例子加以说明。图 6-52 (a) 显示了 TCP 头。在单向数据流中前后两个连续段有一些字段总是相同的，图中用阴影标识了这样的字段。发送端传输实体所要做的只是：将 5 个字从原型头复制到输出缓冲区中，然后填充下一个序号（从内存中的一个字复制过来）、计算校验和、递增内存中的序号；然后将头和数据交给一个特殊的 IP 过程，由它来发送一个常规的、最大的段。紧接着，IP 将它 5 个字的原型头（如图 6-52 (b)）复制到缓冲区中，然后填充标识（Identification）字段、计算校验和。现在这个数据包已经可以发送出去了。

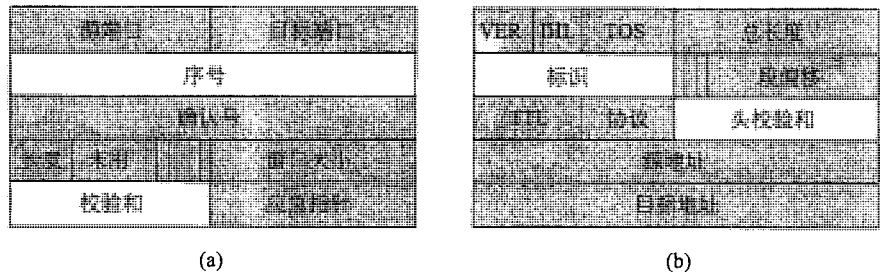


图 6-52
(a) TCP 头; (b) IP 头
这两种情况下它们都取自原型, 没有改变

现在让我们来看图 6-51 中接收端的快速路径处理过程。第 1 步是找到与入境段相对应的连接记录。对于 TCP, 连接记录可能被保存在一张哈希表中, 该哈希表把两个 IP 地址和两个端口作为关键字执行某个简单函数。一旦在该表中找到相应的连接记录, 则比较两个地址和两个端口, 以便验证找到了正确的连接记录。

加速连接记录查询过程的一种优化措施是维护一个指向最近使用过的连接记录的指针, 并且在开始查找时首先使用它。(Clark 等, 1989) 试用了这种方法, 发现它的命中率超过 90%。

找到了连接记录后, 接收端就检查入境段, 看它是否正常: 连接状态是 ESTABLISHED、连接的两端均不打算关闭该连接、段完好无缺、段没有特殊的标志位, 并且它的序号正好是接收端所期望的。这些测试都是一些指令。如果所有的条件都满足, 则调用一个特殊的快速路径 TCP 过程。

快速路径更新连接记录, 并且将数据复制给用户。在复制数据时, 它同时计算校验和, 从而消除了对数据的再次遍历操作。如果校验和正确, 则更新连接记录, 并返回发送端一个确认。这种首先快速检查头然后再详细处理的通用模式称为头预测 (header prediction), 即首先快速地检查段的头, 确定该段是否自己所期望的段, 然后再调用一个特殊的过程来处理。许多 TCP 实现都使用了这种方法。当这种优化方案与本章中讨论的其他优化方案一起使用后, 有可能使 TCP 的运行速度达到本地内存到内存复制速度的 90%, 这里假设网络本身的速度足够快。

两个方面的性能有可能获得较大幅度的提高, 它们分别是缓冲区管理和计时器管理。缓冲区管理中的主要问题是避免不必要的复制操作, 正如我们上面介绍过的那样。计时器管理非常重要, 因为几乎所有的计时器设置都不会超时。设置计时器是为了防止段的丢失, 但是, 大多数段和它们的确认都会正确地到达目的地。因此, 在计时器很少出现超时的情况下, 优化计时器的管理尤为重要。

一种常见的方案是用链表结构将计时器事件按超时值进行排序保存。链表中的头表项包含一个计数器, 指明了离超时时间还有多少个滴答数。每个后续表项也包含一个计数器, 指明在前一个表项超时后还有多少个滴答。因此, 如果 3 个计时器分别在 3、10 和 12 个滴答之后超时, 则 3 个计数器分别为 3、7 和 2。

每滴答一次, 表头表项中的计数器被减 1。当它减到 0 的时候, 它的事件被处理, 链表中的下一个表项变成链表的头。头表项中的计数器值不用改变。在这种方案中, 插入和

删除计时器是很昂贵的操作，其执行时间正比于链表的长度。

如果最大的计时器间隔有上界，并且可以预先知道，那么我们可以使用一种更加高效的方法。这里，要用到一个称为计时轮（timing wheel）的数组，如图 6-53 所示。每个槽对应于一个时钟滴答。图中显示的当前时间为 $T=4$ 。三个计时器分别在从现在开始的 3、10 和 12 个滴答时调度。如果一个新的计时器突然被设置成在 7 个滴答后超时，那么只需在第 11 个槽中加入一个表项即可。类似地，如果要取消一个设置在 $T+10$ 超时的计时器，则只要搜索第 14 个槽中开始的链表，并将所请求的表项移除即可。请注意，图 6-53 中的数组不可能容纳超过 $T+15$ 的计时器。

当时钟滴答时，当前时间指针向前移动一个槽（循环移动）。如果现在指向的表项不为 0，则该表项中的所有计时器都要被处理。（Varghese 和 Lauch, 1987）讨论了基于这个基本思想的许多变种方法。

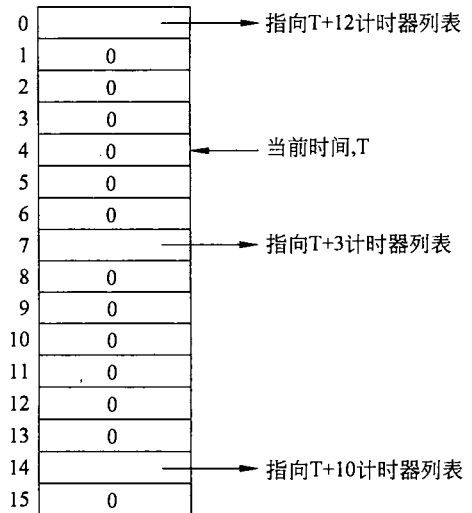


图 6-53 计时轮

6.6.5 头压缩

我们很长以来一直在考查高速网络，那里还有更多的内容需要关注。现在让我们考虑无线网络和其他一些网络的性能，这些网络的带宽非常有限。降低软件开销有助于移动计算机运行得更加高效，但当网络链路成为瓶颈时，它对提高性能却是无能为力。

为了更好地使用带宽，协议头和有效载荷应该携带尽可能少的比特。对于有效载荷来说，这意味着必须使用信息的压缩编码，比如 JPEG 格式的图像，而不是位图或者诸如包含了压缩内容 PDF 的文档格式。这也意味着应用程序级的缓存机制应该摆在首位，比如用降低传输的 Web 缓存机制。

协议头怎么样呢？在无线网络中的链路层，典型的头是压缩的，因为它们始终秉承着专为稀缺带宽而设计的思想。例如，802.16 头采用了短的连接标识符，而不是长长的地址。然而，高层协议，比如 IP、TCP 和 UDP 在所有链路层上采用的是同一版本，而且它们不是专为紧凑压缩头设计的。事实上，为减少软件开销而采用的流水型处理往往导致头无法压缩，而本来是可以做到的（例如，IPv6 有一个比 IPv4 更松散的头）。

高层协议的头是一个重要的性能因素。考虑 IP 语音数据，它通过 IP、UDP 和 RTP 几个协议的组合传输。这些协议需要 40 个字节长的头（IPv4 头 20 字节，UDP 协议 8 字节和 RTP 协议头 12）。IPv6 的情况更糟：需要的头长达 60 个字节，其中包括 40 个字节的 IPv6 报头。头占据了传输数据的大部分，并且消耗了一半多的带宽。

头压缩（header compression）技术可用来降低高层协议头消耗的链路带宽，通常采用专门设计的方案，而不是通用的方法。这是因为头一般都很短，所以它们单独压缩的效果不那么好，而且解压缩时要求之前的所有数据都已经收到。显然，如果一个数据包丢失，

就无法满足这个要求。

使用协议格式的知识来压缩头可以获取巨大的收益。第一个方案由（Van Jacobson, 1990）设计，对通过低速串行链路上的 TCP/IP 头进行压缩。它能够将一个典型的 40 个字节的 TCP/IP 头压缩到平均 3 个字节。这种方法的技巧如图 6-52 给出。一个数据包与另一个数据包之间的许多头字段并没有改变。因为没有必要改变，例如，发送的每一个数据包中具有相同的 IP TTL 字段或相同的 TCP 端口号字段。这些字段可以在链路的发送端省略，在链路的另一端接收端填充上。

类似地，其他字段以可预见的方式改变着。例如，除非丢失，TCP 序号随着数据发送而向前推进。在这种情况下，接收端可以预测到可能的值。即使如此，它可以是先前值的一个很小的变化，因为随着在相反方向收到的新数据，确认号也随之在递增。

采用头压缩，高层协议的头就有可能非常简单，而且可以在低带宽链路上进行压缩编码。鲁棒头压缩（ROHC, RObust Header Compression）是头压缩的现代版本，作为一个框架由 RFC 5795 定义。它的设计目标是能够容忍发生在无线链路上的丢失。每一组被压缩的协议都有一个轮廓（profile）文件，比如 IP/UDP/RTP。最终传输的压缩的头是携带一个指向上下文的引用，它本质上是一个连接；对于同一个连接上的数据包，其头中的字段可以很容易地预测出来，但不同的连接上的数据包头字段则无法预测。在典型的操作中，ROHC 将 IP/UDP/RTP 头从 40 个字节经过压缩降低到 1~3 个字节。

虽然头压缩的目标主要针对的是减少带宽需求，但它也有助于降低延迟。延迟由传播延迟和传输延迟两部分数据组成，前者对于给定的网络路径而言是固定的，而后者则取决于带宽和发送的数据量。例如，在 1 Mbps 链路上发送 1 比特需要 1 微秒。在无线网络的介质上，网络速度相对较慢，因此传输延迟成为整体延迟中的一个重要因素，而且低延迟对于服务质量一贯重要。

头压缩技术有助于减少发送的数据，从而降低传输延迟。发送较小的数据包，可以达到同样的效果。为了降低传输延迟，将增加软件开销。请注意，延迟的另一个潜在来源是访问无线链路的排队延迟。这个延迟也很显著，因为作为网络中的有限资源，无线链路往往被过度地使用。在这种情况下，无线链路必须具备服务质量机制，给实时数据包以较低的延迟。仅仅进行头压缩是不够的。

6.6.6 长肥网络的协议

20 世纪 90 年代初，出现了长距离传输数据的千兆网络。由于结合了快速网络或“肥管道”（fat pipeline）和长延迟，这些网络称为长肥网络（long fat networks）。当这些网络出现时，人们的最初反应是使用已有的协议，但是很快各种各样的问题随之出现。在这一小节中，我们将讨论随着网络协议的速度和延迟尺度增加会出现的一些问题。

第一个问题是许多协议使用了 32 位长的序号。当 Internet 刚开始时，路由器之间的线路主要是 56 kbps 的租用线路，所以一台全速发送数据的主机需要一星期的时间才可能用完一轮序号（即导致序号回绕）。对于 TCP 的设计者们来说， 2^{32} 是一个相当体面的无穷大近似数，因为一周以前发送的老数据包仍然停留在网络中的危险性几乎不存在；对于 10 Mbps 以太网，序号回绕时间变成了 57 分钟，尽管时间更短了，但仍然可以管理；对于

1 Gbps 以太网, 如果主机全速发送数据到 Internet 上, 则序号回绕时间大约是 34 秒钟, 大大低于 Internet 上的最大数据包生存期, 即 120 秒。突然间, 2^{32} 不再是一个良好的无穷大近似数。而且, 当老的数据包仍然存在网络中时, 快速发送端就开始循环使用序号空间了。

问题在于许多协议的设计者简单地认为使用完整序号空间所需要的时间大大超过包的最大生存期, 而且这种假设还没有声明。因此, 没有必要甚至不用担心序号回绕时老的重复数据包仍然存在这个问题。但是对千兆位的速度, 这种未声明的假设无疑是失败的。幸运的是, 已经证明可以通过采用时间戳来扩展有效的序号范围, 这个时间戳作为 TCP 头的选项作为以高序列比特的形式被每个数据包携带着。这种机制称为防止序号回绕 (PAWS, Protection Against Wrapped Sequence numbers), 在 RFC 1323 文档中有详细描述。

第二个问题是流量控制窗口的大小必须增长得很快。例如, 考虑从圣地亚哥发送一个 64KB 的突发数据到波士顿, 以便填满接收端的 64 KB 缓冲区。假设链路速率 1 Gbps, 光在光纤中的单向延迟为 20 毫秒。最初, 在 $t=0$, 管道是空的, 如图 6-54 (a) 所示。500 微秒后, 如图 6-54 (b) 所示所有的段都被发送到光纤上了。前面的段现在出现在 Brawley 附件的某个地方, 仍然位于南加州的腹地。然而, 发送端必须停止, 直到它收到一个窗口更新段。

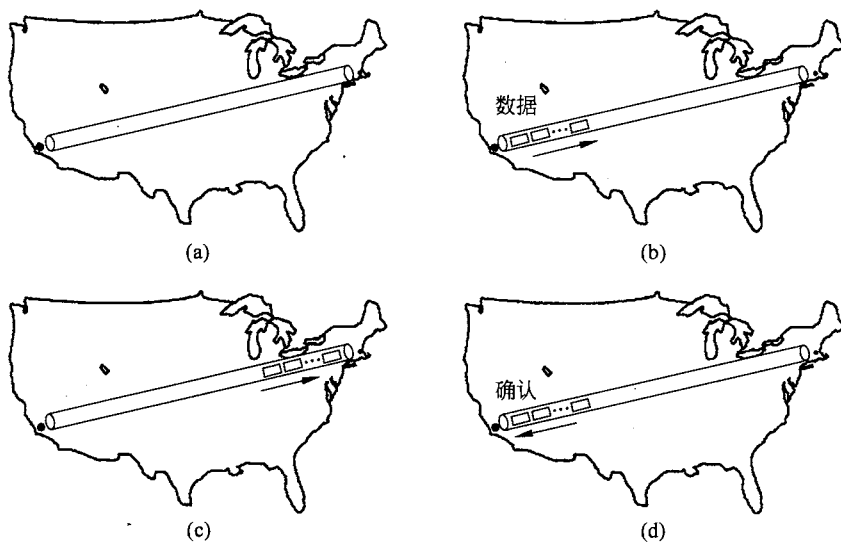


图 6-54 从圣地亚哥给波士顿发送 1 Mb 的状态
(a) 在 $t=0$ 时; (b) 500 微秒之后; (c) 20 毫秒之后; (d) 40 毫秒之后

20 毫秒后, 先遣段到达波士顿, 如图 6-54 (c) 所示, 并得到确认。最后, 在开始发送后 40 毫秒, 第一个确认返回到发送端, 然后开始发送第二次突发数据块。由于传输线路只使用了 100 中的 1.25 毫秒, 因此其效率大约是 1.25%。这是老协议运行在千兆线路上的典型情况。

在分析网络性能时要时刻牢记的一个有用数值——带宽延迟乘积。它是往返延迟时间 (秒) 乘以带宽 (比特/秒)。这个乘积就是从发送端到接收端并且返回的管道的容量 (比特)。

对于图 6-54 的例子来说, 带宽延迟乘积是 40 个百兆比特。换句话说, 发送端必须突发 40 个百兆的比特才能在第一个确认返回前保持全速传输。也就是说, 需要这么多个位来填充管道 (在两个方向上)。这就是为什么突发百兆位的一半只能达到 1.25% 的效率: 它仅为管道容量的 1.25%。

这里可以得出的结论是为了获得良好的性能，接收端的窗口必须至少和带宽时延积一样大，最好再大点，因为接收端可能无法立即响应。对于一个跨洲的千兆线路，需要至少 5 MB。

第三个相关问题是简单重传策略，比如回退 N 协议，在一条带宽延迟乘积非常大的线路上性能非常差。考虑一条 1 Gbps 横贯大陆的线路，其往返传输时间是 40 毫秒。在这段时间中发送端可以传输 5 MB 的数据。如果一个错误被接收端检测出来，那么 40 毫秒以后发送端才能接到错误通知。如果使用回退 N 协议，则发送端不仅要重传坏的数据包，而且要重传该数据包后面的 5 MB 数据。很明显，这是资源的极大浪费，因此需要更复杂的协议，例如选择重传协议。

第四个问题是千兆位线路与百兆位线路在本质上的不同，长距离的千兆位线路的制约因素是延迟而不是带宽。在图 6-55 中，我们看到将一个 1 Mb 文件以不同的传输速度传输 4000 千米所需要的时间。在速度增加到 1 Mbps 以前，传输时间主要受到发送速率的制约；超过 1 Gbps 时，40 毫秒的往返延迟时间远远超过了将数据发送到光纤上所需要的 1 毫秒时间。此时，进一步增加带宽也不会有任何帮助。

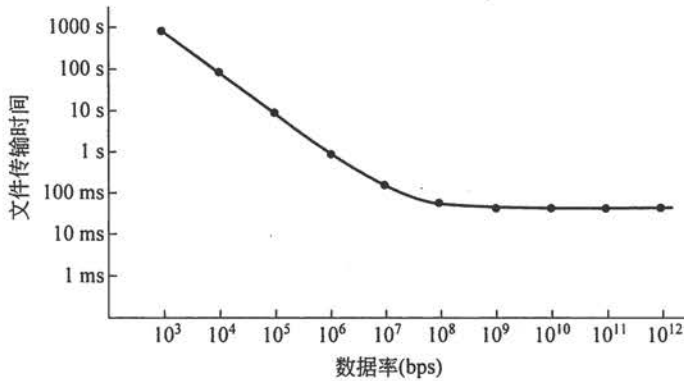


图 6-55 在一条 4000 公里长的线路上发送 1M 位文件需要的传送和确认时间

图 6-55 对于网络协议有一些不幸的影响。从图中可以看出，比如像 RPC 这样的停-等式协议其性能有一个固有的上界。这种限制来自于光速的限制，光学的技术再怎么进步也无法改善这种状况（不过，新的物理法则可能会有所帮助）。除非在一台主机等待应答时千兆线路可以用作一些其他用途，否则千兆线路比百兆线路没什么好的，只是更昂贵而已。

第五个问题是通信速度已经得到很大提高，远远超越了计算速度（计算机工程师们请注意：冲出去，击败那些通信工程师！我们都指望你了）。在 20 世纪 70 年代，ARPANET 只能运行在 56 kbps 的线路上，而当时的计算机已经达到 1MIPS 的计算速度。将这些数字与在 1 Gbps 线路上交换数据包的 1000MIPS 计算机进行比较。每个字节的指令数量已经下降了超过 10 倍。确切的数字值得商榷，具体依赖于测试日期和场景，但由此得出的结论是：协议处理的可用时间少于过去协议处理所用的时间，因此协议必须变得更加简洁。

现在让我们从存在的问题转到处理这些问题的办法上来。所有高速网络设计者必须牢记在心的基本原则是：

为速度而不是为带宽优化进行设计。

老协议的设计目标通常是最小化线路上传输的比特个数，一般的做法是使用小的字段，

并且将这些小的字段组合成字节或者字。这种关注对于无线网络依然有效，但对于千兆位的快速网络不再适用。协议处理是个问题，所以，协议的设计重点变成如何将协议本身最小化。显然，IPv6 的设计者们清楚这条原则。

一种很吸引人的加速办法是用硬件来构建快速网络接口。这种策略的困难在于除非协议极其简单，否则，硬件就是带有第二个 CPU 和自己独立程序的一块插卡。为了确保网络协处理器 (network coprocessor) 比主 CPU 便宜，它通常是一块慢速芯片。这种设计的直接结果是主 (快速) CPU 的许多时间是空闲的，它要等待第二个 (慢速) CPU 完成关键性的工作。认为主 CPU 在等待期间可以去其他工作的想法是不切实际的。而且，当两个通用 CPU 相互通信时，可能会产生竞争条件，所以两个处理器之间的通信协议需要精心设计，以便保证两者能正确地同步并且避免竞争。一般来说，最好的途径是使协议尽可能的简单，并且让主 CPU 来完成协议处理工作。

数据包的布局结构也是千兆网络中一个很重要的考虑因素。头应该尽可能包含少的字段，以便减少包的处理时间，而且头中的这些字段应该足够大到正常工作，另外，这些字段应该按字对齐以便于快速处理。在这里的上下文中，“足够大”意味着类似于下面的问题不会发生：比如序号回绕时老的数据包仍然存在，或者由于窗口字段太小导致接收端不能宣告足够大的窗口空间。

为了降低软件开销和有效操作，最大的数据长度应该非常大。对于高速网络来说，1500 个字节太小，这就是为什么千兆以太网支持大到 9KB 的巨型数据帧以及 IPv6 支持大于 64 KB 的大块数据包传输的理由。

现在来看高速协议中的反馈问题。由于 (相对) 较长时间的延迟回路，应该尽可能地避免反馈：接收端给发送端发送信号需要太长的时间。第一个反馈例子是利用滑动窗口协议来控制传输速率。为了避免由于接收端给发送端发送窗口更新信息而造成固有的 (长时间) 延迟，未来的协议最好使用基于速率的协议，这样才能避免接收端发送窗口更新段给发送端所带来的固有的 (长) 延迟。在这样的协议中，发送端可以发送所有它希望传送的数据，只要发送速率不超过双方预先协商好的速率即可。

第二个反馈例子是 Jacobson 的慢速启动算法。这个算法需要执行多次探测来确定网络的处理能力。对于高速网络，执行几次小的探测才能确定网络的响应能力将浪费大量带宽。一种更加有效的方法是让发送端、接收端和网络三者在建立连接时都预留下必要的资源。提前预留资源还有另一个好处，就是更加容易减少抖动。简而言之，网络越是向高速发展，设计就越是被无情地推向面向连接的操作，或者非常接近于面向连接的操作。

另一个有价值的特性是在连接请求过程中顺带发送正常量数据的能力。按照这种方法，可以节省一次往返时间。

6.7 延迟容忍网络

我们将通过描述一种全新的传输来结束本章，这种传输有朝一日可能会成为 Internet 的重要组成部分。TCP 和大多数其他传输协议建立在这样的假设基础之上：发送端和接收端通过一些工作路径持续地连接在一起，否则协议就会失败且数据将无法传递。在某些网

络中，往往没有端到端的路径。一个例子是空间网络，比如低地球轨道（LEO, Low-Earth Orbit）卫星网络，卫星频繁进出地面站的探测范围。一个给定的卫星或许只能在特定的时间与地面站通信，而两颗卫星可能在任何时间都永远也无法彼此沟通，甚至通过一个地面站作中转也无法直接通信，因为其中一颗卫星可能总是超出地面站范围。其他类似的网络涉及潜艇、公共汽车、移动电话和其他具有计算机的设备，由于移动性或极端条件的限制，使得它们只有间歇性的连通性。

在这些偶尔连接的网络中，可以把数据存储在某些节点，并在随后有工作链路时再转发这些数据，因此数据通信仍然可以进行，这种技术称为消息交换（message switching）。最终数据将被中继到目的地。基于这种通信方法构建的网络体系结构称为延迟容忍网络或中断容忍网络（DTN, Delay-Tolerant Network, or a Disruption-Tolerant Network）。

DTN 的工作始于 2002 年，当时 IETF 成立了一个研究小组来研究这个主题。DTN 的灵感来自一个不太可能的源头：努力在太空中发送数据包。空间网络必须处理间歇性的通信和很长的延迟。Kevin Fall 观察到这些星际互联网的思想可以应用到地球上那些间歇性连接呈现常态的网络（Fall, 2003）。该模型给出了 Internet 的一个有用的概括，即在 Internet 上通信期间可以进行存储和延迟。数据传输是类似于邮政系统或者电子邮件的传递，而不是路由器上的数据包交换。

自 2002 年以来，DTN 体系结构已经经过了细化，DTN 模型的应用也有了很大增长。作为主流应用，考虑由科学实验、媒体事件或基于 Web 服务生成的许多 TB 级的大量数据集，需要将这些数据集复制到位于世界各地不同地点的数据中心。运营商希望在非高峰时间发送这大块的流量，以便使用已经支付但没有被使用的带宽，并且愿意容忍一些延迟。它喜欢在晚上做备份，当其他应用程序没有大量使用网络时。问题在于对于全球性的服务，非高峰时间在世界各地不同的位置是不同的。波士顿和珀斯数据中心的非高峰期网络带宽在时间上很少有重叠，因为一个城市的夜晚是另一个城市的白天。

然而，DTN 模型允许在数据传输过程中进行存储和延迟。有了这个模型，就有可能使用非高峰期的带宽把数据集从波士顿发送到阿姆斯特丹，因为这两个城市的时差仅相隔 6 个小时的时区；然后将数据集存储在阿姆斯特丹，直到阿姆斯特丹和珀斯之间有非高峰期带宽可用；然后将数据集发送到珀斯，完成这次数据传送。（Laoutaris 等，2009）对该模型进行研究，并发现它可以较小的代价提供很大的容量；与传统的端到端模型相比，经常使用 DTN 模型通常能将容量增加一倍。

下面我们将描述 IETF DTN 体系结构和协议。

6.7.1 DTN 体系结构

Internet 的主要假设是在整个通信会话期间源端和接收方之间存在一条端到端路径，这正是 DTN 寻求释放的条件。当情况并非如此时，正常的 Internet 协议就会失败。DTN 能避开这种缺乏端到端连接这个问题，关键在于它采用了如图 6-56 所示的基于消息交换的体系结构。DTN 的目的还在于容忍低可靠性和大延迟的链路。DTN 的体系结构由 RFC 4838 说明。

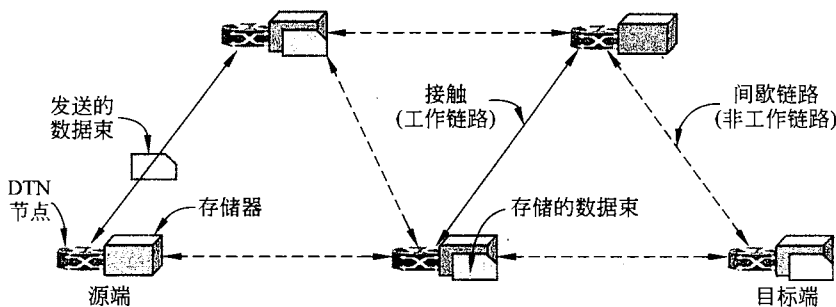


图 6-56 延迟容忍网络体系结构

按照 DTN 的术语，一条消息称为一个数据束（bundle）。DTN 节点都配备了存储器，典型的是持久性存储介质，比如磁盘或闪存。它们将数据束存储起来，直到链路变得可用，然后转发该束。链路的工作呈现间歇性特点。图 6-56 显示了 5 条当前都没有在工作的间歇性链路和两条正在工作的链路。一条工作链路称为一次接触（contact）。图 6-56 还显示了存储在两个 DTN 节点上的数据束，它们正在等待接触的到来以便向前发送。以这样的方式，通过一次次接触数据束从源被中继到它们的目的地。

在 DTN 节点存储和转发包，听起来类似于路由器上的排队和转发数据包，但两者之间有质的区别。在 Internet 的路由器上，排队发生以毫秒计，或最多以秒计。而在 DTN 节点，数据束可存储好几个小时，直到公交车到达小镇、飞机完成一次飞行、一个传感器节点吸收足够的太阳能开始运行、处于睡眠状态的计算机被唤醒等。这些例子还指出了 DTN 节点存储的第二个区别，那就是存储了数据的节点可能会移动（比如公交车或飞机），而且这种运动甚至是数据传递的关键部分。Internet 上的路由器不允许移动。移动数据束的整个过程可能更好地称为“存储-携带-转发”。

作为一个例子，考虑如图 6-57 所示的场景，这是在空中第一次使用 DTN 协议（Wood 等，2008）。数据束源是一种 LEO（低地球轨道卫星），它正在记录地球图像，作为灾害监测星座卫星的一部分。记录下来的图像必须返回给采集点。然而，卫星在绕地球旋转时只能间歇性地接触三个地面站。它依次与每个地面站进入接触。卫星、地面站和收集点每个都作为一个 DTN 节点。在每一次接触时，一个数据束（或一束的一部分）被发送到地面站；然后经过骨干陆地网络，数据束被发送到采集点，由此完成地球图像的传送。

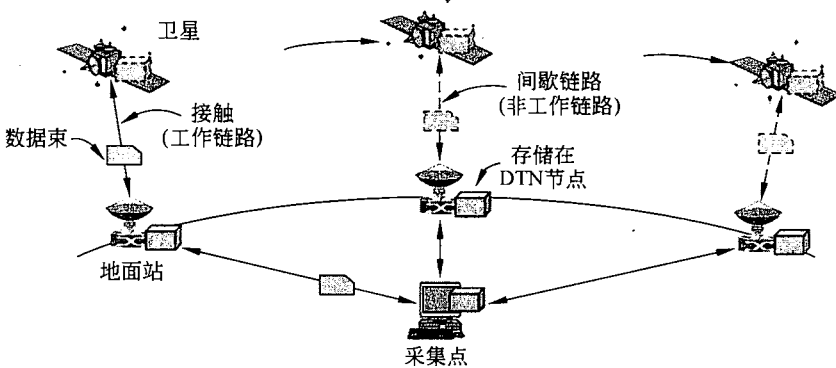


图 6-57 DTN 在空中的应用

在这个例子中，DTN 体系结构的主要优点是它自然而然适应了卫星需要存储图像的情形，因为在图像拍摄时根本没有连接。除此之外，在这种应用场景中该体系还有另外两个优点。首先，恐怕不存在一个单一的接触，持续时间足够长到发送完整个图像。然而，它们可以分散到与三个地面站的接触上。其次，使用卫星和地面站的接触可以将链路骨干网络彻底脱钩。这意味着卫星的下载不会受到慢速地面链路的限制。它可以全速率地处理，数据束存储在地面站上，直到它可以中继给采集点。

这个体系结构没有说明的一个重要问题是如何寻找一条穿过 DTN 节点的良好路径。良好的路由取决于描述何时发送数据的体系结构性质，也取决于采用哪次接触。有些接触在使用之前就已经知道。一个很好的例子是在空间的天体运动。对于太空实验而言，早就知道何时会发生接触，每经过一个地面站的接触时间间隔为 5~14 分钟，下行链路的容量为 8.134 Mbps。有了这方面的知识，可以提前计划传输一束图像的路径。

在其他情况下，接触也可以预测，但是缺少确定性。这样的例子包括公交车，由于事先有时间表，通常公交车以最常规的方式与每个车站接触；类似地，虽然有些变化，ISP 网络的非高峰时间和带宽也可以从过去的历史中预测出来。另一种极端情况是接触完全是偶然和随机的。一个例子是用户通过移动电话传递数据，那么这种接触就取决于在一天之中哪些用户彼此接触。当存在着不可预测的接触时，可采用这样的路由策略：沿着不同的路径发送数据束的副本，希望在束的生存期到达之前其中的一个副本能被传递到目的地。

6.7.2 数据束协议

为了更清楚地了解 DTN 操作，我们现在来考查 IETF 协议。DTN 是一种新兴的网络，因而没有使用 IETF 协议的需求，所有实验性质的 DTN 都使用了不同的协议。然而，这些协议至少是我们开始和突出许多关键问题的良好的开端。

DTN 协议栈如图 6-58 所示。关键的协议是数据束协议（Bundle protocol），该协议由 RFC 5050 描述。它负责接受来自应用程序的消息，并通过“存储-携带-转发”操作将这些数据作为一个或者多个数据束发送到目标 DTN 节点。从图 6-58 中很明显地看出，数据束协议运行在 TCP/IP 层之上。换句话说，TCP/IP 可以被用在 DTN 节点之间的每个接触上，把数据束从接触一端的 DTN 节点移动到另一端的 DTN 节点。这种协议定位提出了数据束协议究竟是一个传输层协议还是应用层协议这样一个问题。正如 RTP 一样，我们采取这样的立场，尽管它运行在传输协议之上，由于束协议为许多不同的应用提供了传输服务，所以我们在本章覆盖 DTN 网络。

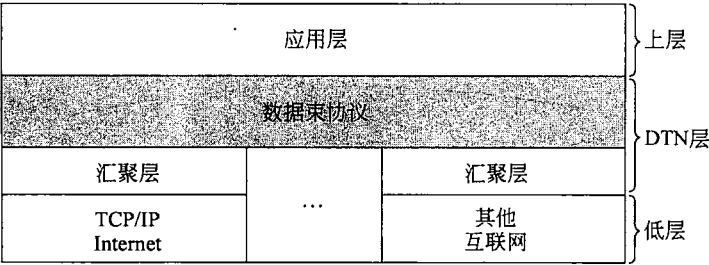


图 6-58 延迟容忍网络协议栈

在图 6-58 中我们看到，数据束协议可能运行在其他协议之上，比如 UDP，或者甚至其他种类的互联网协议之上。例如，在一个空间网络中链路可能有很长的延迟。地球和火星之间的往返时间很轻易地达到 20 分钟，这取决于行星的相对位置。可以设想一下，在这样的链路上 TCP 确认和重传将如何工作，尤其是针对相对短的消息。它根本无法工作。相反，另一种使用纠错码的协议可能会有作用。或者在资源受到非常约束的传感器网络中，可以使用一个比 TCP 更轻量级的协议。

由于数据束协议是固定的，但它主要运行在各种传输协议之上，因此在功能上协议之间必定留有沟壑。这种沟壑正是在图 6-58 中引入汇聚层的原因。汇聚层就像是匹配所连协议接口的粘胶层。根据定义，针对每个不同的底层传输协议，有一个不同的汇聚层。汇聚层通常是在标准中加入新的和现有协议。

数据束协议的消息格式如图 6-59 所示。这些消息中的不同字段告诉我们一些由数据束协议处理的关键问题。

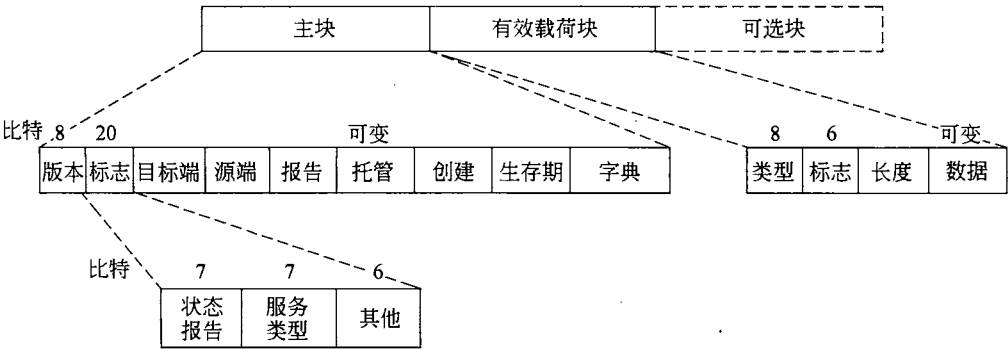


图 6-59 数据束协议的消息格式

每个消息由一个主块（Primary block）、一个有效载荷块（Payload block）和一个可选块（Optional block）组成。主块可以被看作是头，有效载荷包含着数据，而其他块可以携带一些安全参数。主块始于版本（Version）字段（当前为 6），紧接着一个标志（Flags）字段。在其他功能中，标志位编码了一类服务，让源端把它的数据束标记成更高或更低的优先级，以及诸如接收方是否应该确认该数据束等其他处理请求。

然后是地址字段，其中突出了 3 个有趣的设计部分。除了目标端（Destination）和源端（Source）标识符字段外，还有一个托管（Custodian）标识符。托管是负责照看包交付的当事人。在 Internet 中源节点通常就是托管人，因为如果数据没有最终交付到目的地，那么它就是重发的节点。然而，在 DTN 中，源节点可能并不总能连接上，而且可能没有办法知道这些数据是否已经传递到目的地。DTN 使用了托管传送（custody transfer）来处理这个问题。在托管传送方式中，另一个接近接收方的节点假设承担照看数据安全传递的责任。例如，如果一个包裹被装在飞机上，在稍后时间和地点被转发，那么飞机就成为包裹的托管人。

第二个有趣的方面是这些标识符不是 IP 地址。由于数据束协议是为了能在跨越各种传输协议和互联网上工作，它定义了自己的标识符，这些标识符实际上比诸如 IP 地址这样的底层地址更像个高层名称，如网页的 URL。它们给 DTN 提供了应用程序级的路由，比如

电子邮件传送或软件更新分发。

第三个有趣的方面是标识符的编码方式。还有一个报告（Report）标识符用于诊断消息。所有的标识符被编码为一个指向可变长度字典（Dictionary）字段的引用。当托管人或报告节点与源或目标节点相同时，这种方式就提供了压缩。事实上，消息的格式设计牢记着可扩展性和效率，因而使用了可变长度字段的压缩表示法。压缩表示法对于无线链路和资源受限的节点（比如传感器网络）至关重要。

接下来是一个创建（Creation）字段，该字段携带数据束的创建时间、从远端开始排序的序号以及生存期（Lifetime）字段，生存期给出了该束数据的有效时间。这些字段必须给出，因为数据可能被存储在 DTN 节点上相当一段时间，必须有某种方式从网络中删除过时的数据。与 Internet 不同的是，它们需要 DTN 节点有松散同步的时钟。

主块的结尾是字典（Dictionary）字段；接着是有效载荷块，该块由一个简短的类型（Type）字段开始，标识了它的有效载荷；接着是一些标志（Flag）字段，描述了如何处理该数据束的选项；然后是数据（Data）字段，之前是长度（Length）字段。最后，还有可能有其他可选块，比如携带安全参数的块。

有关 DTN 的许多方面正处于学术研究界的探索之中。良好的路由策略取决于接触的性质，正如上面所述的那样。存储在网络内部的数据提出了其他问题。现在拥塞控制必须考虑节点上的存储器，因为存储器变成了另一种可耗尽的资源。缺乏端到端的通信同时加剧了安全问题。在 DTN 节点成为数据束的托管人之前，它要知道发送端是否授权使用网络，并且数据束是否是接收方想要的。这些问题的解决方案将取决于 DTN 的类型，因为空间网络与传感器网络是完全不同的两种网络。

6.8 本章总结

传输层是理解分层协议的关键。它提供了各种服务，其中最重要的服务是一个从发送端至接收端的端到端的、可靠的、面向连接的字节流。通过一组服务原语可以访问此服务，原语可用来建立、使用和释放连接。Berkeley 套接字提供了一个常用的传输层接口。传输协议必须能在不可靠的网络上完成连接管理工作。由于延迟的重复数据包可能会重新出现在不恰当的时刻，因而连接的建立过程非常复杂。为了处理这些问题，需要使用三次握手法来建立连接。释放连接比建立连接要容易得多，但由于存在两军对垒问题，释放连接也并非轻而易举。

即使网络层完全可靠，传输层也有大量的工作要做。它必须处理所有的服务原语、管理连接和计时器、拥塞控制与分配带宽以及运行大小可变的滑动窗口流量控制。

拥塞控制应该在竞争流之间公平地分配所有可用带宽，并跟踪网络使用的变化。AIMD 控制法则能收敛到公平和有效的带宽分配。

Internet 有两个主要的传输协议：UDP 和 TCP。UDP 是一个无连接的协议，它主要对 IP 数据包进行了包装，由此引入了多个进程复用和分用一个 IP 地址这一特性。UDP 可以被用于客户机-服务器之间的交互（比如 RPC），UDP 也可以被用来建立实时协议（比如 RTP）。

主要的 Internet 传输协议是 TCP。它提供了一个可靠的、双向的、拥塞可控的字节流，